SRE Guild Lead

TCG Digital — Interview Handbook

Dev Factory · Guild leadership model (no direct reports) · 10–15 years experience

Round 1: 14 sections, 123 questions — technical & guild practice

Round 2: 7 sections, 58 questions — greenfield, leadership & fit

Quick Reference: 11 sections, 111 questions — precise, sayable L2 answers

Questions marked **[KEY]** are the highest-probability ones worth over-preparing.

Contents

Round 1 — Technical & Guild Practice

Guild Leadership & Influence Without Authority, Observability & Golden Signals, SLIs, SLOs & Error Budgets — Governed Across Pods, Incident Management & Blameless Postmortems, Kubernetes & Cloud-Native Infrastructure, Infrastructure as Code & Toil Reduction, Scripting for Automation — Python, Go, Shell, CI/CD & Release Engineering, Capacity Planning, Performance & Cost, Chaos Engineering, DR & Multi-Region Resilience, Guild Operations — Runbooks, Metrics & Career Pathing, Cross-Guild Collaboration, Scenario & Troubleshooting Bank, Behavioral, Positioning & Closing

Round 2 — Greenfield, Leadership & Fit

Greenfield — Building the SRE Practice From Zero, Executive & Strategic Leadership, Client-Facing & Business Acumen, Deeper Behavioral & Leadership Philosophy, Culture, Values & Fit, HR, Compensation & Logistics, Round 2 — Closing Strong

Quick Reference — L2 Rapid-Fire Answers (ChatGPT-predicted, precise & short)

Guild Leadership & Influence, SLO & Error Budget, Observability, Incident Management, Kubernetes & Cloud Scenarios, IaC / Terraform / Automation, CI/CD & Release Engineering, DR / Chaos / Resilience, Capacity / Cost / Performance, Architecture & NFR, Leadership & Behavioural

Round 1

Technical & Guild Practice · 14 sections · 123 questions

Guild Leadership & Influence Without Authority

The flagship section. Essential Skills line one is "able to set and land standards across pods without formal reporting authority" — expect this tested from multiple angles, not just as one behavioral question at the end.

GUIL-01 What does "guild" actually mean here, and how is it different from your reporting line?

A guild is a **practice community**, not an org chart box. It groups people around a discipline (reliability) that cuts across the delivery structure (pods, which are themselves organized by client/product under "Factory" units at TCG). The model echoes what Spotify popularized with guilds and chapters: chapters are your functional home, guilds are cross-cutting communities of practice that share standards and learning across chapter/team boundaries.

Practically: engineers doing SRE/DevOps work sit embedded *inside* pods and report to their pod lead or delivery manager. You don't own their headcount, their performance review, or their sprint priorities. What you own is the **practice** — the standards, tooling patterns, and craft those engineers are expected to apply, wherever they sit.

GUIL-02 How would you write the SRE guild charter for TCG Digital?

Keep it to one page, because a charter nobody reads is not a charter. I'd structure it as:

- **Purpose** — why the guild exists (consistent reliability practice regardless of which pod or client engagement a service sits under)
- **Scope** — what's in (SLOs, incident process, observability standards, IaC/automation patterns) vs. explicitly out (day-to-day pod delivery priorities, which stay with pod leads)
- **Non-negotiables** — the small set of standards every pod must meet (e.g., every production service has an owner, an SLO, and an on-call path) vs. recommended patterns pods can adapt
- **Engagement model** — guild sync cadence, how a pod raises an exception, how the guild escalates a gap to delivery leadership
- **Success metrics** — the guild health numbers you'll track and report

The charter is a starting draft, not a decree — I'd socialize it with pod leads before it's final, because buy-in on the non-negotiables list is the actual hard part.

GUIL-03 A pod lead says "we don't have time for your observability standard this sprint." What do you do?

I don't escalate first — I reframe first. Their real objection is usually "this is extra work with no payoff for me," so I'd connect the standard to something they already feel: fewer 2am pages, faster root-causing when something breaks on *their* service, less time spent explaining incidents to the client. If I can show a golden-path template that gets them 80% of the standard with minutes of setup (a Helm chart with dashboards/alerts pre-wired), the friction argument usually disappears.

If they still say no, I ask what they'd need to fit it in — sometimes it's genuinely a sequencing problem, not a values problem, and I can pair with one engineer for an afternoon instead of asking the whole pod to context-switch. Escalation to delivery leadership is a last resort, reserved for a pattern of refusal, not a single sprint's pushback.

GUIL-04 How is a guild lead different from an engineering manager, in your own words?

An EM's authority is positional — headcount, reviews, sprint priorities. A guild lead's authority is earned and constantly re-earned — through technical credibility, the usefulness of what you publish, and whether pods actually get value from engaging with you. If I stop being useful, pods stop showing up to guild syncs; there's no org chart forcing them to. That's a sharper incentive to stay hands-on than most management roles carry.

It also changes what "success" looks like: an EM's team ships what the EM prioritizes. A guild's success is measured by *adoption* — how many pods actually run the standard, not how many were told to.

GUIL-05 How would you run an effective guild sync?

Short, regular, and artifact-producing — not a status meeting. My default cadence: 30 minutes, biweekly, standing agenda:

– One incident learning from the last two weeks (what pattern should every pod know about)
– One standard proposal or update, open for pushback
– One tooling demo or golden-path improvement (5 minutes, rotating presenter across pods — this is also how you build embedded champions)
– Open floor for any pod's blocker

Everything discussed gets written down in the guild's shared space the same day — if it's not written down, it didn't happen, and the next pod that hits the same question has to ask again instead of finding an answer.

GUIL-06 What artifacts does a healthy guild actually maintain?

– The charter and standards/RFCs (versioned, with a lightweight "propose -> discuss -> adopt" process)
– Shared runbooks and playbooks, indexed by symptom, not just by service
– Golden-path templates (Helm charts, Terraform modules, CI/CD pipeline templates) that make the standard the path of least resistance
– A guild health dashboard (SLO attainment, MTTR trend, automation coverage, standard-adoption rate across pods)
– An onboarding doc for new pods joining the guild's scope

GUIL-07 How do you build technical credibility with engineers who don't report to you?

By showing up where the work actually happens, not just where the standards get published. Concretely: joining a pod's incident retro even when I wasn't paged, opening a PR against their repo instead of just filing a suggestion, being reachable during someone else's on-call shift the first time they use a new runbook I wrote. Credibility with senior engineers is earned in the trenches, not in the charter document.

The fastest way to lose it is publishing a "standard" that's clearly never been run against a real production service — so before I'd propose anything guild-wide, I'd pilot it myself against one real pod first.

GUIL-08 Tell me about a time you influenced a technical decision without formal authority over the people involved.

This is the question this whole role hinges on, so don't wing it — structure it in STAR (Situation, Task, Action, Result), and pull a real example from your 14+ years, ideally one where you moved a team toward a practice or tool (Kubernetes adoption, a Terraform/IaC standard, an observability change) without having line authority over the people you convinced.

What the interviewer is scoring: did you lead with data/credibility or with insistence, did you adapt when you hit resistance, and did the change actually stick after you moved on to the next thing. A guild lead who can point to one practice that "runs itself" now, months after they stopped pushing it, is exactly the signal this role wants.

> *Tip: **Prep note:** pick a real, specific story from Barclays or Flyingedge/Russell Investments before tomorrow, and rehearse the numbers you actually remember — don't reach for a number you're not sure of live in the room.*

GUIL-09 Two pods want conflicting standards — one wants Datadog, one wants OSS Prometheus/Grafana for cost reasons. How do you decide?

I wouldn't force one tool guild-wide unless there's a real reason to (shared on-call tooling, a client compliance requirement, or genuine guild-wide cost pressure). What I'd standardize is the **outcome** — every pod exposes the four golden signals, every pod's alerts route through the same on-call/escalation path, every pod's SLO dashboard follows the same layout convention — and let implementation vary if the outcome is equivalent.

Over-standardizing tooling when the pods have genuinely different constraints (client-mandated tools, existing investment, team familiarity) is how guilds lose credibility. Standardize the contract, not the vendor, unless there's a concrete reason the vendor itself needs to be uniform.

GUIL-10 You don't own headcount, but the JD says you contribute to hiring and career pathing for the SRE discipline. How?

I'd build a shared SRE competency rubric (junior -> senior -> staff-equivalent, by skill area) that pod leads use when writing job descriptions and doing performance calibration for SRE-flavored roles — so "senior SRE" means the same thing whether the pod is in Kolkata or Hyderabad. I'd sit on interview panels for SRE-titled roles across pods as the technical-bar-setter, and I'd offer a light mentoring track for engineers in a pod who want to grow into SRE work, even though they don't report to me.

The value to delivery leadership: without this, every pod invents its own bar for "SRE," and quality becomes inconsistent across client engagements — which is exactly the risk a services company like TCG Digital can't afford.

GUIL-11 How would you track and report "guild health" to Dev Factory leadership?

A small, honest dashboard beats a big vanity one. I'd report: incident trend (volume + severity mix over time), MTTR trend, SLO attainment rollup across pods, and automation/toil-reduction coverage (% of repetitive ops work that's been self-serviced or automated). I'd present it monthly, with one paragraph of "what changed and why" rather than just numbers — leadership needs the story, not just the chart.

I'd resist the urge to report "adoption %" as a vanity metric alone — a pod that adopted the standard on paper but never actually alerts on it isn't a win, so I'd pair adoption with an outcome metric (did their MTTR actually improve).

GUIL-12 You're the senior escalation point for a critical incident in a pod using a stack you don't know deeply. What do you actually add?

Not deep service knowledge — that's the pod's job. What I add is incident command discipline: keeping one person driving, one person scribing, making sure someone's actually watching the clock against the SLO burn rate, and asking the cross-cutting questions the pod is too close to ask themselves ("did anything else deploy in this window," "has this exact symptom happened on another pod before"). Pattern recognition across pods is the genuinely unique value a guild lead brings to an incident they don't own day-to-day.

I also protect the pod from stakeholder noise during the incident, so their engineers can stay heads-down fixing it instead of fielding status requests.

GUIL-13 How do you avoid becoming a bottleneck if every pod routes reliability questions through you?

Documentation-first and self-service by default — if I answer the same question twice, the second answer becomes a runbook entry, not a repeat conversation. I'd deliberately cultivate an embedded "champion" in each pod (not a formal role, just the engineer who's absorbed the most guild context) so questions get answered locally most of the time, and I only get pulled in for genuinely novel problems.

If I notice I'm the single point of failure for a decision that should be routine, that's a signal the standard isn't documented well enough yet — not a reason to keep being the bottleneck.

GUIL-14 An embedded SRE/DevOps engineer in a pod isn't following guild standards. You have no authority to discipline them — what's your move?

Coach first, and assume good faith — usually it's a constraint I don't know about yet (a client deadline, a legacy system that doesn't fit the standard cleanly), not defiance. I'd have a direct, low-drama conversation, understand the blocker, and either help solve it or adjust the standard if it genuinely doesn't fit their context.

If it's a repeated pattern with no real constraint behind it, I'd loop in their pod lead — not to get someone in trouble, but because sustained non-adoption is a signal the standard itself needs the pod lead's buy-in, which is their call to give, not mine to force.

GUIL-15 In one sentence — what's your leadership style, and how does it translate to a role with zero direct reports?

Answer this with something true about how you've actually operated across 14+ years as a senior IC/technical lead — most senior SREs already lead this way day-to-day (setting a technical direction other engineers follow without a formal reporting line), they just haven't had to name it as a discipline before. The honest framing: "I've been doing guild-style leadership inside teams for years; this role just makes it the explicit job instead of a side effect of being the senior person in the room."

> *Tip: **Prep note:** write your own one-sentence version tonight and say it out loud once before the interview — a leadership answer that sounds rehearsed-but-not-memorized lands far better than one you're composing live.*

Observability & Golden Signals

Named tools: Prometheus, Grafana, ELK/OpenSearch, Datadog. The JD explicitly calls out "golden signals" — that's Google SRE Book language, and a strong signal the interviewer is fluent in that framework.

OBS-01 Define the four golden signals with a concrete threshold example for each.

– **Latency** — time to serve a request. Split success vs. error latency separately (a fast error is not "good latency"). Example: p99 < 300ms for checkout.

– **Traffic** — demand on the system, e.g. requests/sec or queue depth. Baseline it so you can spot anomalies, not just absolute spikes.
– **Errors** — rate of failed requests, explicit (5xx) and implicit (200 with wrong content). Example: error rate > 1% over 5 minutes pages.
– **Saturation** — how "full" the system is (CPU, memory, connection pool, queue). Example: alert at 80% of a hard resource ceiling, before you hit it.

OBS-02 Monitoring vs. observability — what's the actual difference, and why do people conflate them?

Monitoring answers questions you thought to ask in advance — dashboards and alerts built around known failure modes. Observability is the property that lets you answer questions you *didn't* anticipate, by exploring high-cardinality data after the fact. The three pillars — metrics, logs, traces — are the raw material; observability is what you can do with them when a novel failure shows up.

They get conflated because good monitoring is built on observable systems, but you can have excellent dashboards for known failure modes and still be blind the first time something genuinely new breaks — that gap is exactly what "observability" as a discipline exists to close.

OBS-03 You have pods on Datadog and pods on OSS Prometheus/Grafana. How do you standardize observability across them without forcing a tool migration?

Standardize the contract, not the backend: every service exposes the same four golden signals, under the same naming convention, alertable through the same on-call routing regardless of which backend stores the data. I'd publish a "golden signal spec" (metric names, label conventions, alert severity mapping) that both a Prometheus exporter and a Datadog agent config can satisfy.

Where I would push for consolidation is the *on-call and escalation layer* — everyone should page through one path (e.g., a shared PagerDuty/Opsgenie integration) even if the metrics live in different backends, because inconsistent paging is where guild-wide incident response actually breaks down.

OBS-04 Write a PromQL alert for error rate above 1% over 5 minutes.

```
- alert: HighErrorRate
  expr: |
    sum(rate(http_requests_total{status=~"5.."}[5m]))
    /
    sum(rate(http_requests_total[5m]))
    > 0.01
  for: 5m
  labels:
    severity: page
  annotations:
    summary: "Error rate above 1% for 5m on {{ $labels.job }}"
    runbook: "https://guild.internal/runbooks/high-error-rate"
```

Note the `for: 5m` — it prevents paging on a single noisy scrape; the condition has to hold for the full window. I'd always attach a runbook link in the annotation — an alert without a runbook is a guaranteed 2am Google search.

OBS-05 Where does ELK/OpenSearch fit next to a metrics stack like Prometheus/Grafana?

Metrics tell you *that* something's wrong and roughly where; logs tell you *why*, at the level of an individual request or stack trace. I'd reach for Prometheus/Grafana first for alerting and trend analysis (cheap to store, fast to query, built for numeric time series), and drop into OpenSearch/ELK when I need to search unstructured detail — a specific error message, a correlation ID across services, an audit trail.

OpenSearch is the AWS-driven, Apache-2.0-licensed continuation of the pre-license-change Elasticsearch/Kibana stack — functionally similar to ELK for most SRE use cases, and the more common choice now for teams that don't want Elastic's licensing terms.

OBS-06 What makes an alert good instead of noisy?

– Symptom-based (user-facing impact), not cause-based — alert on "checkout is slow," not "CPU is at 70%" (that's a diagnostic signal, not a page)
– Actionable — if there's nothing a human can do at 2am, it shouldn't page; it can log or ticket instead
– Has a runbook link
– Distinguishes page-worthy (customer impact, now) from ticket-worthy (needs attention, not urgent)
– Tuned against real burn rate, not a static threshold that pages on every minor blip

Alert fatigue is a reliability risk in itself — a guild-wide review of alert-to-page ratio per pod is a good recurring guild health check.

OBS-07 How do you design dashboards that work for both engineers and non-technical delivery leadership?

Two layers, not one dashboard trying to do both jobs. Engineers get a drill-down view: golden signals per service, saturation detail, recent deploys overlaid on the graph. Leadership gets a rollup: SLO attainment as a single number/color per client engagement, incident count trend, nothing that requires reading a PromQL query to interpret. I'd link the leadership view down into the engineering view for anyone who wants to go deeper, rather than maintaining two disconnected dashboards.

OBS-08 What causes cardinality explosion in Prometheus, and how do you fix it?

Usually an unbounded label value — putting a user ID, request ID, or raw URL path into a metric label instead of a bounded value like a route template or status class. Each unique label combination is a new time series, and Prometheus's memory/query cost scales with that. Fix: move high-cardinality data to logs/traces instead of metric labels, use route templates ("/users/:id") instead of literal paths, and set up a cardinality-limiting relabel rule or recording rule to catch regressions before they take down the TSDB.

OBS-09 What does "dashboards and alerts as code" look like in practice?

Grafana dashboards defined as JSON (or via a tool like Grafonnet/Jsonnet for reuse) and committed to git, synced into Grafana via its provisioning API or a sidecar rather than clicked together in the UI. Alert rules live as YAML next to the service's other IaC. The payoff for a guild specifically: a golden-path dashboard template a new pod can fork and get 90% of the standard instantly, with changes reviewable in a PR instead of silently drifting in someone's browser session.

OBS-10 How would you handle log isolation and cost control for a multi-tenant, multi-client observability setup?

Since TCG Digital's pods often serve different client engagements, log isolation isn't just tidiness — it can be a contractual/compliance requirement. I'd use index-per-pod or index-per-client naming conventions in OpenSearch with role-based access scoped accordingly, tiered retention (hot for recent, cold/archived for compliance-driven longer retention), and sampling on high-volume, low-value log lines (health checks, debug-level noise) to keep ingest cost proportional to actual signal.

OBS-11 What would a guild-provided "golden path" observability starter kit look like?

A Helm chart or Terraform module a pod applies to a new service that wires up, by default: the four golden signal dashboards, a standard alert set (error rate, latency SLO burn, saturation), log shipping to the shared OpenSearch cluster with the pod's index namespace pre-configured, and a runbook stub. The goal is that adopting the guild standard is *less* work than not adopting it — that's the only way voluntary adoption scales past a handful of pods.

OBS-12 A pod's dashboards look healthy but customers are complaining. What's your first move?

Check what the dashboard is actually measuring versus what the customer is experiencing — a classic gap is server-side metrics looking clean while the problem is client-side (a CDN, a third-party API, DNS, or a mobile app version). I'd pull real user monitoring or synthetic checks from outside the infrastructure if they exist, and if they don't, that's itself the finding: the observability gap is between "what we instrument" and "what the user experiences," and closing it becomes the actual follow-up work.

SLIs, SLOs & Error Budgets — Governed Across Pods

The JD asks for someone who defines AND operationalizes SLOs "to drive engineering and release decisions" — across pods with different products and clients. The governance angle, not just the textbook definitions, is what separates this from an IC-level answer.

SLO-01 Define SLI, SLO, SLA, and error budget with one example that ties them together.

- **SLI** — the measured indicator, e.g. "% of requests served in under 300ms"
- **SLO** — the internal target for that SLI, e.g. "99.9% of requests under 300ms, over a 30-day window"
- **SLA** — the external, often contractual commitment, usually looser than the SLO on purpose, with a penalty if breached
- **Error budget** — the allowed unreliability: 100% - SLO. At 99.9%, that's ~43 minutes of budget per 30 days. Spend it on risk (releases, experiments); once it's gone, the policy shifts toward stability work.

SLO-02 A pod has never done SLOs before. How do you help them define their first one?

Start narrow and user-facing, not comprehensive. I'd pick the single metric that most directly reflects whether a real user had a good experience — usually availability or latency on the primary user journey (checkout, login, the core API call) — and resist the urge to instrument everything on day one. A pod's first SLO should take an afternoon to define and a week to start getting real data on, not a quarter-long project.

I'd also set the initial target from *observed* historical performance, not an aspirational number pulled from nowhere — if they're actually running at 99.5% today, starting the SLO at 99.99% guarantees an immediately blown budget and kills buy-in on day one.

SLO-03 What actually happens when a pod's error budget is exhausted?

The policy I'd propose guild-wide: once budget hits zero, feature releases pause except for changes that directly improve reliability, until the budget recovers above a re-entry threshold (not necessarily back to 100% — that's too strict and creates its own gaming incentive). It needs teeth — if "budget exhausted" never actually changes anyone's behavior, the whole SLO exercise is theater. The pod lead and delivery lead need to agree to this policy *before* the first time it triggers, or it becomes a fight in the moment instead of a pre-agreed process.

SLO-04 How do you use SLOs to drive an actual release go/no-go decision?

Before a release: check current burn rate. If the budget's healthy, ship normally. If it's tight, either delay, ship behind a flag to a small percentage first, or require an explicit sign-off from the pod lead acknowledging the risk. After: watch burn rate in the minutes/hours post-deploy as a primary rollback signal — a fast-burning budget after a deploy is often a clearer, earlier signal than waiting for a support ticket to land.

SLO-05 How do you standardize SLO practice across pods serving completely different products and clients?

You don't standardize the *numbers* — a batch data pipeline and a customer-facing checkout API have nothing in common numerically. You standardize the *process*: every production service must have at least one SLO, defined using the guild's template, reviewed quarterly, reported through the same rollup format, with an error budget policy that's actually enforced. That's the governance layer a guild lead owns; the specific target and SLI stay with the pod that knows their service and their client's actual needs best.

SLO-06 What is multi-window, multi-burn-rate alerting, and why bother?

Instead of one static threshold, you alert on the *rate* at which budget is being consumed, checked over multiple windows (e.g., 5 minutes and 1 hour) simultaneously. A fast burn over a short window catches a sudden outage quickly; a slower burn checked over a longer window catches a persistent, lower-grade degradation that a short-window check would miss entirely. Google's SRE workbook popularized the standard 2%/5%/10% budget-consumption thresholds mapped to different urgency levels — worth knowing that specific framework by name if it comes up.

SLO-07 What does a good SLO rollup look like when reporting to Dev Factory leadership across many pods?

One row per pod/service: current attainment vs. target, budget remaining, trend arrow, and a one-line "why" if it's red. Leadership doesn't need the PromQL; they need to know which client engagements are at reliability risk this month and whether that risk is improving or worsening. I'd flag persistent red rows as a guild-level intervention candidate, not just a pod problem to quietly track.

SLO-08 A client contractually demands an SLA that's unrealistic given the current architecture. How do you handle it?

This is a guild-to-delivery-leadership conversation, not something an engineer should be left holding alone. I'd translate the SLA into the internal SLO it implies, show the gap against current measured reality with data, and give delivery/account leadership the actual tradeoffs: what architecture investment would close the gap, what it costs, and what the risk is if we commit anyway without it. My job is to make the risk visible and quantified — the business call on whether to accept that risk with the client isn't mine to make alone, but it's absolutely mine to surface clearly and early, not after a breach.

SLO-09 What's the practical engineering-effort difference between a 99.9% and a 99.95% SLO?

99.9% allows ~43 minutes of downtime per month; 99.95% allows ~21.6 minutes — roughly half. That last increment usually costs disproportionately more: multi-AZ isn't enough anymore, you need multi-region or true zero-downtime deploy patterns, more sophisticated retry/circuit-breaker logic, and often a fundamentally different on-call response time commitment. I'd push back hard on any pod or client defaulting to "five nines" without discussing what that actually costs to build and staff — most services genuinely don't need it, and chasing it is often waste, not rigor.

SLO-10 How do you set an achievable starting SLO for a chronically unreliable legacy service?

Set it at (or just slightly above) current measured performance, not at an industry-standard target. The point of an SLO here isn't aspiration, it's making the *current* reliability visible and giving the team a real, honest baseline to improve from. I'd pair it with an explicit improvement roadmap and revisit the target quarterly as reliability work actually lands — an SLO the team already knows they're failing on day one just gets ignored.

SLO-11 How would you tie error budgets to how pods allocate engineering time?

As a guild-wide policy recommendation, not a mandate I can enforce directly: when a service is consistently burning its full budget, that's a signal the pod should allocate a fixed share of the next sprint (I'd suggest a ballpark like 10–20%, tuned per pod) to reliability work before taking on new feature scope. I'd frame this to pod leads as protecting their own velocity, not slowing it — an unreliable service generates more interrupt-driven firefighting than the reliability work would have cost, it's just less visible in a sprint plan.

Incident Management & Blameless Postmortems

JD language: "oncall design, escalation paths, severity classification, and blameless postmortems," plus you're explicitly the "senior escalation point for critical production incidents."

INC-01 How would you design on-call rotations across pods with different sizes and a mixed onshore/offshore setup?

No single rotation guild-wide — each pod owns its own primary/secondary rotation, sized to their team (a 3-person pod can't sustain the same rotation cadence as a 12-person one without burning people out). What I would standardize is the *handoff and escalation contract*: a documented handoff checklist, a maximum acceptable time-to-acknowledge per severity, and — given TCG's onshore/offshore blend — explicit time-zone-aware routing so a Sev1 doesn't sit waiting for someone who's asleep when a follow-the-sun handoff was available.

I'd also push for a minimum team size before a pod runs its own rotation at all (below a certain size, on-call burnout risk outweighs the benefit of pod-local ownership, and it may be better to pool with a neighboring pod).

INC-02 Walk through a severity classification framework with response-time expectations.

- **Sev1** — full outage or critical data issue, customer-facing. Page immediately, 15-min ack, all-hands-on-deck, guild lead pulled in as senior escalation.
- **Sev2** — major degradation, partial impact. Page, 30-min ack, pod handles with pod lead aware.
- **Sev3** — minor impact, workaround exists. Ticket, next business day.
- **Sev4** — cosmetic/no user impact. Backlog.

The exact thresholds matter less than the discipline of classifying *consistently* across pods — a "Sev1" in one pod and a "Sev2" for the identical symptom in another erodes the whole framework's credibility with leadership.

INC-03 Design an escalation path — who gets paged, and when does the guild lead get pulled in?

Primary on-call -> secondary (if no ack in X minutes) -> pod lead (for Sev1/Sev2, for awareness and any business-impact decisions) -> guild lead (for Sev1s past a duration threshold, cross-pod incidents, or when the pod's own on-call is stuck and needs fresh eyes / cross-pod pattern-matching). I'd make the "guild lead gets paged" trigger explicit and automatic — e.g., any Sev1 unresolved past 30 minutes — rather than leaving it to someone's judgment call mid-incident, because that's exactly the moment judgment is most impaired.

INC-04 What actually makes a postmortem "blameless" — not just labeled that way?

- Timeline of **events and decisions**, not people's names attached to mistakes ("the deploy at 14:02 introduced X," not "Priya's deploy broke X")
- Facilitator explicitly reframes any blame-flavored comment in the room, in real time
- Root cause analysis goes past the human action to the systemic gap that allowed it — if a person "should have caught it," ask why the system didn't catch it for them
- Action items target process/tooling, not "be more careful"
- Leadership visibly doesn't punish the person who caused the incident — this has to be demonstrated, not just stated, or nobody believes the "blameless" label

INC-05 Walk through a Five Whys example.

Symptom: checkout API returned 500s for 12 minutes.

Action item lands on the process gap (add the checklist item / lint rule), not on "the engineer should have tested timeouts" — that's the difference between Five Whys done well and done as a blame exercise with extra steps.

INC-06 You're pulled into a critical incident in a pod's stack you barely know. What do you do in the first 15 minutes?

I don't try to become the domain expert in 15 minutes — I run the incident, and let the pod be the domain expert. Concretely: confirm someone's driving the technical fix and someone's scribing the timeline, ask the cross-cutting questions the pod may be too deep in the weeds to ask ("what changed in the last hour — deploys, config, traffic pattern," "has this symptom shown up on another pod recently"), and start tracking SLO burn rate against a mitigation deadline so we know when to consider a rollback versus continuing to chase root cause live.

My value in those 15 minutes is process and pattern-matching, not stack-specific knowledge — and being explicit about that with the pod up front avoids me getting in the way of people who know the system better than I do.

INC-07 What is the incident commander role, concretely, during a live Sev1?

Not necessarily the person fixing the issue — the person coordinating everyone who is. Responsibilities: keep a single source of truth for status, decide when to escalate or pull in more people, manage stakeholder communication so engineers aren't interrupted mid-fix, make the call on rollback vs. continue-forward, and declare when the incident is actually resolved (not just "looks better"). On a cross-pod incident, the guild lead is a natural fit for this role since nobody in a single pod has the standing to coordinate across pod boundaries.

INC-08 How do you actually reduce MTTR across pods, not just report the number?

- Better alerting (symptom-based, with runbook links) cuts time-to-detect
- Standardized dashboards cut time-to-diagnose — engineers aren't relearning a new dashboard layout per incident
- Rehearsed rollback procedures (not "we could roll back" but "here's the one command") cut time-to-mitigate
- A guild-maintained "common failure patterns" doc cuts time-to-root-cause for repeat-shaped incidents

MTTR is a lagging indicator of all of the above being in place — chasing the number directly without fixing the inputs just produces gamed reporting.

INC-09 What does a good runbook actually contain, and how do you keep it from going stale?

Symptom (how you'd recognize this from an alert), diagnosis steps in order, the actual fix commands (not "restart the service" — the real command, with the right namespace/flags), rollback steps if the fix doesn't work, and escalation contact if it's beyond the runbook's scope. Staleness prevention: every runbook gets used-and-verified during its incident, or reviewed at a fixed cadence (quarterly) with an owner accountable for it — an unowned runbook rots within two quarters, reliably.

INC-10 Postmortem action items keep piling up unresolved. How do you fix that guild-wide?

Treat unresolved action items as a guild health metric, reported alongside MTTR and SLO attainment — visibility alone fixes a surprising amount of this. Beyond that: every action item needs an owner and a due date at the moment it's created (not "someone should"), items should be sized small enough to actually fit in a sprint, and I'd propose a rule that a pod can't have more than N open postmortem actions before it's flagged in the guild health review — forcing prioritization instead of infinite backlog growth.

INC-11 A shared platform dependency breaks and multiple pods are affected simultaneously. How do you coordinate?

This is exactly the cross-pod scenario the guild-lead escalation point exists for. I'd immediately establish a single incident channel spanning all affected pods (not N separate pod-local incidents running in parallel with no shared visibility), assign one incident commander — likely me, since I'm the only person with standing across all affected pods — and have each pod report impact and status into the shared channel rather than fixing in isolation. Root cause ownership goes to whoever owns the shared dependency (possibly the DevOps & Infra guild, if it's platform infrastructure), while each pod handles their own service-level mitigation in parallel.

INC-12 The same root cause keeps recurring across incidents in one pod. What's the guild-level move?

A single postmortem action item clearly isn't fixing it, so I'd escalate this to a guild-level intervention rather than letting it stay a pod-local backlog item: pull the pattern together across incidents (showing it's not three unrelated issues but one systemic gap), bring it to the pod lead with that data, and offer hands-on help — pairing, a focused reliability sprint, or bringing in the Architecture & Design guild if it's a design-level gap rather than an operational one. Recurring incidents are one of the clearest signals that guild-level coordination is adding real value over pod-local fixes.

Kubernetes & Cloud-Native Infrastructure

JD lists AWS, Azure, or GCP — kept concise here since deep Kubernetes drilling almost certainly belongs in your existing K8s-focused handbooks; this section stays scoped to what's specific to a guild-wide, multi-cloud, multi-client context.

K8S-01 A pod's pod (the Kubernetes kind) is stuck in CrashLoopBackOff. Walk through your triage.

`kubectl describe pod` for events first (image pull failure, OOMKilled, failed readiness probe), then `kubectl logs --previous` to see the crash before the restart wiped it. Common root causes in order of frequency: bad config/env var, a missing dependency the app expects at startup, resource limits too tight causing OOMKill, or a readiness/liveness probe misconfigured against a slow-starting app. I'd check resource limits early since it's one of the most common and fastest to confirm.

K8S-02 The JD says AWS, Azure, or GCP — what actually changes operationally between clouds, and what stays the same?

What stays the same: Kubernetes primitives, your golden-signal instrumentation approach, your GitOps/CI-CD model, your IaC discipline. What changes: managed-cluster specifics (EKS IAM/IRSA vs. AKS managed identity vs. GKE workload identity), networking model details (VPC/subnet design differs meaningfully between AWS and Azure), and native observability tooling (CloudWatch/X-Ray vs. Azure Monitor vs. Cloud Operations). Practically, I'd keep the guild's standards cloud-agnostic at the Kubernetes layer and maintain a thin per-cloud adapter for the identity/networking bits that genuinely can't be abstracted away.

> *Tip: **Prep note:** your Azure certifications (AZ-104, SC-300, SC-100) plus hands-on AWS experience map well onto this exact "AWS, Azure, or GCP" line — this JD is one of the few where the multi-cloud framing works in your favor rather than against it. Say so directly if asked about it.*

K8S-03 How would you approach multi-tenancy for pods sharing guild-managed Kubernetes infrastructure?

Namespace-per-pod (or per-client, where isolation needs to be stricter than per-pod) as the baseline, with ResourceQuotas and LimitRanges enforced per namespace to prevent noisy-neighbor resource starvation. NetworkPolicies to restrict cross-namespace traffic to only what's explicitly needed. For clients with real compliance requirements, dedicated node pools or even dedicated clusters may be justified — the guild's job is defining that decision framework (when does a client need cluster-level isolation vs. namespace-level), not deciding it unilaterally per case.

K8S-04 How do you design RBAC when many pods share guild-managed clusters?

Role per pod, scoped to their namespace(s) only, granted via RoleBindings rather than ClusterRoleBindings wherever possible. The guild/platform team holds cluster-admin; pod engineers get namespace-scoped edit access; a narrower read-only ClusterRole for cross-namespace observability tooling. I'd tie role assignment to the same identity provider used org-wide (leaning on the SC-300/Azure AD-style setup if that's the org's IdP) rather than maintaining Kubernetes-native RBAC as a separate source of truth.

K8S-05 How would Helm serve as the guild's golden-path delivery mechanism?

A shared base chart (or library chart) encoding the guild's non-negotiables — resource requests/limits set, liveness/readiness probes required, standard labels for cost attribution and observability, default PodDisruptionBudget — that pods extend rather than reinvent per service. New pods start from `helm create --starter guild-baseline` equivalent instead of a blank chart, so the standard is the default, not an afterthought someone has to remember to bolt on.

K8S-06 How do you approach a guild-wide cluster upgrade without breaking dozens of pods' workloads?

Staged rollout: a non-production or shared staging cluster first, a canary production cluster (or one low-risk pod's namespace) second, then the rest — never all pods on the first attempt. I'd communicate the upgrade window and any deprecated API versions well in advance (`kubectl` deprecation warnings and `pluto`/`kubent`-style scanning catch most breakage before it happens), and I'd own the guild-wide comms so no pod is surprised by an upgrade they didn't know was coming.

K8S-07 How would you give pod-level cost visibility on shared Kubernetes infrastructure?

Consistent cost-attribution labels enforced by the golden-path chart (pod-id, client-id) feeding a chargeback tool like Kubecost or OpenCost, with a per-pod cost dashboard as part of the guild's standard offering. Without enforced labeling at the chart level, cost attribution silently rots the first time someone forgets a label on a new service — so I'd make the label a schema-validated requirement, not a convention people are expected to remember.

K8S-08 Resource requests are set far too low across several pods' services, causing scheduling and stability issues. How do you fix it at guild scale?

Rather than manually auditing every pod's YAML, I'd point VPA (Vertical Pod Autoscaler) in recommendation-only mode at the affected namespaces first to get real usage-based numbers, then propose right-sized requests as a PR against each pod's chart values rather than a top-down mandate — reviewable, low-drama, backed by data. I'd also add a guild-wide lint check (e.g., via OPA/Conftest in CI) that fails a chart missing requests/limits entirely, so this stops recurring for new services.

Infrastructure as Code & Toil Reduction

Terraform, Ansible, Helm — "with a strong toil-reduction mindset" per the JD. Kept concise since deep Terraform drilling likely belongs in your dedicated Terraform handbook; this stays focused on the guild-scale/governance angle.

IAC-01 How would you design reusable Terraform modules that many pods can consume?

A small set of guild-maintained modules for the patterns every pod needs (a standard VPC/network baseline, an EKS/AKS node group pattern, an RDS-with-backups pattern) published from a guild-owned repo and versioned with semantic tags, so pods pin to a known-good version rather than pulling `main` and inheriting breaking changes unexpectedly. Modules should expose sane defaults (encryption on, tagging enforced, logging on) with narrow, deliberate points of customization — not forty optional variables that let a pod accidentally disable every guardrail.

IAC-02 How do you handle Terraform state management across many pods without it becoming a mess?

Separate state per pod (or per pod-per-environment), never one shared monolithic state file — a single state file across dozens of pods turns every apply into a blast-radius risk and a lock-contention bottleneck. Remote backend (S3+DynamoDB, or Azure equivalent) with state files namespaced by pod and environment, and the guild owns the backend infrastructure itself (bucket, lock table, access policy) so pods don't each reinvent that plumbing.

IAC-03 Terraform vs. Ansible — where does each fit in a guild-standardized toolchain?

Terraform for declarative infrastructure provisioning — the "what should exist" layer (VPCs, clusters, databases, IAM). Ansible for configuration and orchestration on things that already exist — the "how do I configure this box/app" layer, and for one-off operational tasks that don't fit Terraform's declarative model well. I wouldn't force every pod onto both; if a pod is fully containerized on Kubernetes, they may have little need for Ansible at all, and mandating it anyway is exactly the kind of over-standardization that erodes guild credibility.

IAC-04 Define "toil" per the Google SRE book, and how would you measure it across pods guild-wide?

Toil is manual, repetitive, automatable, tactical work that scales linearly with service growth and carries no enduring value once done. I'd have each pod self-report (roughly) what share of their operational time is toil during a quarterly guild check-in, track the trend rather than chase precision, and prioritize guild-level automation investment toward whichever toil pattern shows up identically across the *most* pods — that's where a single guild-built tool gets the most leverage.

IAC-05 Show a toil-reduction script example — Docker image cleanup.

```
#!/usr/bin/env bash
set -euo pipefail

# Remove dangling and unused images/containers older than 14 days (336h)
docker image prune -af --filter "until=336h"
docker container prune -f --filter "until=336h"

echo "Toil cleanup complete"
```

`set -euo pipefail` is the non-negotiable first line of any production bash script — exit on error, exit on unset variable, and fail a pipeline if any stage fails, not just the last one.

IAC-06 What would guild-mandated "policy as code" look like, and how do you enforce it without becoming the bottleneck?

OPA/Conftest (or the Terraform-native Sentinel, cloud-dependent) rules checked in CI before any pod's `terraform apply` — no encryption-disabled S3 buckets, no security groups open to 0.0.0.0/0, mandatory cost-attribution tags. The key to not becoming a bottleneck: these run automatically in each pod's own pipeline, guild-owned as a shared CI template, not as a manual review I personally have to approve every time. Automated guardrails scale; a human approval gate on every pod's infra change does not.

IAC-07 How would you detect and manage Terraform drift at guild scale across dozens of pods' infrastructure?

A scheduled `terraform plan` (read-only, no apply) run against every pod's state on a recurring cadence, with any detected drift surfaced to a shared guild dashboard rather than silently discovered during the next real change. Drift usually means someone made a manual console change under incident pressure — I'd treat repeated drift on the same resource as a signal to either lock that resource down harder or accept it needs a documented manual-change exception, rather than just re-applying and hoping it doesn't happen again.

IAC-08 How do you make Helm the guild's packaging standard without it feeling like a mandate?

Same golden-path logic as everywhere else in this handbook: make the guild-provided chart genuinely the fastest way to ship a compliant service, with real documentation and a working example, and adoption follows because it's easier — not because it's required. I'd track adoption as a guild health metric, but the actual lever is chart quality and pilot success stories from early-adopter pods, not a top-down deadline.

Scripting for Automation — Python, Go, Shell

Go is explicitly named alongside Python and Shell — likely lighter-covered ground than Python/Bash in your other prep, so weighted accordingly here.

CODE-01 Why would an SRE guild specifically reach for Go over Python for tooling?

Distribution is the big one: Go compiles to a single static binary with no runtime/dependency install needed, which matters enormously when you're shipping a guild-wide CLI tool to dozens of pods that may not all have the same Python version or virtualenv discipline. Performance and real concurrency (goroutines) matter for anything doing a lot of parallel I/O, like polling many services' health endpoints. It's also the native language of the ecosystem you're already operating in — Kubernetes, Prometheus, Terraform, Docker are all written in Go, so contributing to or extending those tools' own ecosystem favors Go.

Python still wins for quick scripts, data-shaping, and anything glueing together APIs where developer velocity matters more than distribution or raw performance.

CODE-02 Show a short Go example — a concurrent health checker.

```
package main

import (
■"fmt"
■"net/http"
■"sync"
■"time"
)

func checkHealth(url string, wg *sync.WaitGroup, mu *sync.Mutex, results map[string]bool) {
■defer wg.Done()
■client := http.Client{Timeout: 3 * time.Second}
■resp, err := client.Get(url)
■ok := err == nil && resp.StatusCode == 200
■mu.Lock()
■results[url] = ok
■mu.Unlock()
}

func main() {
■pods := []string{"http://pod-a/healthz", "http://pod-b/healthz", "http://pod-c/healthz"}
■var wg sync.WaitGroup
■var mu sync.Mutex
■results := make(map[string]bool)

■for _, url := range pods {
■■wg.Add(1)
■■go checkHealth(url, &wg, &mu, results)
■}
■wg.Wait()

■for url, ok := range results {
■■fmt.Printf("%s healthy=%v\n", url, ok)
■}
}
```

Each pod's health check runs concurrently via a goroutine, `sync.WaitGroup` waits for all to finish, and a mutex protects the shared results map from concurrent writes — the standard pattern for fan-out/fan-in in Go.

CODE-03 Show a Python example — rolling up SLO burn rate across services for a guild health report.

```
#!/usr/bin/env python3
"""Roll up SLO burn-rate across pods for the weekly guild health report."""
import requests

PROM_URL = "http://prometheus.guild.svc:9090"
SERVICES = ["checkout", "payments", "catalog", "search"]

def burn_rate(service: str) -> float:
    query = (
        f'sum(rate(http_requests_total{{job="{service}",status=~"5.."}}[1h])) '
        f'/ sum(rate(http_requests_total{{job="{service}"}}[1h]))'
    )
    resp = requests.get(f"{PROM_URL}/api/v1/query", params={{"query": query}}, timeout=5)
    result = resp.json()["data"]["result"]
    return float(result[0]["value"][1]) if result else 0.0

for svc in SERVICES:
    rate = burn_rate(svc)
    flag = "AT RISK" if rate > 0.01 else "OK"
    print(f"{svc:12s} error_rate={{rate:.4%}}  {{flag}}")
```

Doubled braces (`{{` / `}}`) inside the f-string are how you escape a literal brace when the string itself also needs f-string interpolation — worth knowing cold if you're live-coding a PromQL query builder.

CODE-04 What does a production-grade Bash script skeleton look like, versus a throwaway one?

```
#!/usr/bin/env bash
set -euo pipefail
trap 'echo "FAILED at line $LINENO" >&2' ERR

LOG_FILE="/var/log/guild/$(basename "$0").log"
log() { echo "[$(date -Is)] $*" | tee -a "$LOG_FILE"; }

log "Starting..."
# ... actual work here ...
log "Done."
```

The difference from a quick one-liner: `set -euo pipefail` for fail-fast behavior, a trap that reports exactly which line failed (invaluable when a guild-wide script fails silently in a pod you don't have eyes on), and structured logging instead of bare `echo` so failures are traceable after the fact, not just in a terminal that's already gone.

CODE-05 When would you actually choose Go over Python for a specific SRE tool, concretely?

If it's going to be distributed as a CLI to every pod (no dependency install friction), if it needs real concurrency at scale (checking hundreds of endpoints in parallel), or if it's extending something already in the Go ecosystem (a kubectl plugin, a Prometheus exporter, a Terraform provider). If it's a one-off analysis script, a quick API glue job, or something only the guild itself runs, Python's faster to write and iterate on — I wouldn't rewrite something in Go just for its own sake.

CODE-06 How would you package and distribute a guild-wide CLI tool to pods that don't all have the same environment?

This is where Go's single-binary distribution pays off directly: a GitHub Actions/GitLab CI release pipeline that cross-compiles for linux/amd64 and arm64, publishes to a shared internal artifact store or GitHub Releases, and a one-line install script (`curl | sh` pattern, or better, a checksum-verified equivalent) pods run once. Versioning matters — pods should be able to pin a known-good version rather than always pulling latest, the same discipline as the Terraform module versioning above.

CODE-07 Once a script becomes guild-wide infrastructure other pods depend on, what changes about your quality bar?

It stops being a personal script and starts being a dependency — so it needs: actual error handling (not just letting exceptions crash silently in someone else's pipeline), a test suite if it's doing anything non-trivial, versioned releases instead of "just pull latest," and documentation good enough that a pod engineer can use it without asking me. I'd treat guild-published tooling with the same code review bar as any production service, even if it "started as a quick script" — the moment other teams depend on it, the throwaway-script mindset becomes a liability.

CI/CD & Release Engineering

Jenkins, GitLab CI, ArgoCD named in the JD. Kept concise — the guild-governance angle (shared templates, standardized gates) is the differentiator over a generic CI/CD Q&A.

CI/C-01 Jenkins vs. GitLab CI vs. ArgoCD — where does each fit, and would you push pods to standardize on one?

Jenkins and GitLab CI are both CI (build/test/package) — which one a pod uses often comes down to what their client engagement or existing toolchain already runs, and I wouldn't force a migration purely for guild tidiness unless there's a real driver (cost, security, maintenance burden). ArgoCD is different in kind — it's CD via GitOps, reconciling cluster state to a git source of truth — and I would push harder to standardize on GitOps for the CD half specifically, because a consistent deployment model across pods pays off directly in incident response speed (everyone knows how to check "what's actually deployed" the same way).

CI/C-02 Explain the App-of-Apps GitOps pattern with ArgoCD for many pods/services.

A root ArgoCD Application whose only job is to manage other Application manifests — one child Application per pod or per service — so onboarding a new pod's GitOps setup is "add one manifest to the root app" rather than manually configuring ArgoCD per service. It scales guild administration cleanly: I can see and reason about every pod's deployed state from one root application tree instead of dozens of disconnected configurations.

CI/C-03 What pipeline security gates would you mandate guild-wide?

SAST (static analysis) on every PR, dependency/SCA scanning for known CVEs, secret scanning to block committed credentials before merge, and container image scanning before anything reaches a registry pods can pull from. I'd make these part of a shared pipeline template rather than something each pod configures independently — consistency here is a security requirement, not just convenience, especially across client engagements with different compliance obligations.

CI/C-04 How do you make canary/blue-green deployment a guild-recommended standard that pods without deep deployment expertise can actually use?

Bake it into the golden-path Helm chart and ArgoCD rollout config rather than expecting each pod to hand-roll their own traffic-shifting logic — Argo Rollouts (the ArgoCD-adjacent controller) supports canary/blue-green natively with analysis-based promotion gates tied directly to the golden-signal metrics from the observability section. A pod adopting the standard chart gets safe deployment patterns "for free," which is exactly the leverage a guild should be optimizing for.

CI/C-05 How would you roll out shared CI/CD pipeline templates without breaking pods mid-flight?

Versioned templates (pods pin a version, same discipline as the Terraform modules), a deprecation window before an old version is retired, and a pilot pod validating any new template version before it's offered guild-wide. I'd never force an in-flight pipeline update on every pod simultaneously — that's a guaranteed guild-credibility hit the first time it breaks someone's release during a client-facing deadline.

CI/C-06 What rollback standard would you push across pods?

Every pod needs a rollback that's a single command or button, rehearsed (not theoretical), and faster than root-causing forward. With GitOps, rollback is usually a git revert of the deployed manifest, which ArgoCD then reconciles automatically — I'd treat "can this pod roll back in under 5 minutes without me on the call" as a real guild-adoption checkbox, not just a nice-to-have.

Capacity Planning, Performance & Cost

CAP-01 How do you approach capacity planning across pods with very different growth trajectories?

Pod-owned forecasting (they know their own client's growth/seasonality far better than I do), guild-provided methodology: trend current utilization against historical growth rate, add a safety margin sized to how volatile that pod's traffic actually is, and review quarterly rather than annually since client engagements can shift fast. I'd standardize the *process and cadence* guild-wide, not the numbers — a payments pod's capacity planning inputs look nothing like a batch-analytics pod's.

CAP-02 Explain the USE method for performance troubleshooting.

For every resource (CPU, memory, disk, network), check **Utilization** (% busy), **Saturation** (queued work waiting), and **Errors** (failed operations). It's a fast, systematic sweep to find the actual bottleneck instead of guessing — I'd pair it with RED (Rate, Errors, Duration) for the request-facing side of a service, since USE covers resources and RED covers the requests flowing through them.

CAP-03 Where would you start a guild-wide cloud cost optimization initiative?

Orphaned/unattached resources first (unattached EBS volumes, idle load balancers, old snapshots) — pure waste with zero risk to fix. Then right-sizing based on actual utilization data rather than guesswork (VPA recommendations on Kubernetes, Compute Optimizer or equivalent for VMs). Then commitment discipline (reserved instances/savings plans for genuinely steady-state baseline load, kept separate from bursty capacity). I'd surface a per-pod cost dashboard before asking anyone to change anything — most cost waste is invisible simply because nobody's ever seen it broken down by pod before.

CAP-04 What load testing approach and tooling would you offer as a guild service?

k6 or Gatling for scriptable, CI-integratable load tests (both are more developer-friendly and version-controllable than JMeter's XML-heavy format, in my experience) — offered as a guild template a pod can point at their own service, with thresholds tied to their actual SLO rather than an arbitrary pass/fail. I'd distinguish load (expected peak), stress (find the breaking point), soak (sustained duration, catches memory leaks), and spike (sudden burst) testing, since pods often only think to ask for one of these when they actually need a different one for their real risk.

CAP-05 Show a simple k6 load test script.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 50 },
    { duration: '5m', target: 50 },
    { duration: '2m', target: 0 },
  ],
  thresholds: {
    http_req_duration: ['p(95)<300'],
    http_req_failed: ['rate<0.01'],
  },
};

export default function () {
  const res = http.get('https://api.example.com/checkout');
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);
}
```

Ramp to 50 virtual users over 2 minutes, hold for 5, ramp down — with thresholds that fail the CI run automatically if p95 latency or error rate breaches the SLO-derived target, turning this into a real release gate rather than a manual one-off test.

CAP-06 How do you balance cost optimization against reliability so you don't cut into error budget headroom?

Never optimize cost against the same margin that's protecting your SLO — e.g., don't shrink autoscaling headroom below what's needed to absorb a real traffic spike just to trim a cloud bill. I'd frame any cost change explicitly against its reliability impact before approving it guild-wide: "this saves $X and costs Yms of p99 headroom" as a joint number, not cost and reliability reviewed by two people who never talk to each other.

Chaos Engineering, DR & Multi-Region Resilience

CHAO-01 Explain the core principles of chaos engineering.

Define a steady-state hypothesis (what "normal" looks like, measurably), inject a real-world failure (kill a pod, add network latency, saturate a resource), and see whether the steady state holds. Critically: control the blast radius (start in staging, or a small % of production traffic), have an automatic abort condition, and treat every finding as an input to a fix, not just a demo. The goal isn't breaking things for its own sake — it's finding weaknesses before an uncontrolled failure finds them for you.

CHAO-02 Compare chaos tooling options briefly — Litmus, Gremlin, Chaos Mesh, AWS FIS.

- **LitmusChaos** — CNCF, Kubernetes-native, free/OSS, good default for a Kubernetes-first guild standard
- **Chaos Mesh** — also Kubernetes-native CNCF project, strong for network-level fault injection
- **Gremlin** — commercial, polished UI, broader than just Kubernetes, easier for less chaos-experienced pods to adopt but a licensing cost across every pod
- **AWS FIS** — native to AWS, integrates cleanly if you're AWS-heavy, less useful for Azure/GCP pods

Given TCG Digital's multi-cloud framing (AWS/Azure/GCP), I'd lean toward a Kubernetes-native option like LitmusChaos as the guild default, since it isn't tied to one cloud provider.

CHAO-03 How would you run a guild-wide "game day" across multiple pods?

Pick one realistic, well-scoped failure scenario (a shared dependency going down, a region failover), run it against a willing pilot pod first in staging with a clear abort button, document what actually happened versus what the runbook assumed, then fix the gaps before offering it as a repeatable guild service other pods can request. I would not mandate every pod run a game day on a fixed guild-wide schedule on day one — that's a fast way to get resentful, half-hearted participation instead of genuine learning.

CHAO-04 How do you standardize DR testing cadence across pods with a guild recommendation?

Every production service needs a documented RTO/RPO and a DR test on a fixed cadence (I'd suggest at minimum annually, more often for anything client-critical) — the guild owns the standard and the reporting, pods own the actual test execution since they know their service's failover mechanics. I'd track "time since last successful DR test" as a guild health metric alongside SLO attainment, since an untested DR plan is really just an assumption.

CHAO-05 Active-active vs. active-passive multi-region — how do you help a pod choose?

Active-passive is simpler and cheaper — one region serves traffic, the other stands ready — but costs you failover time and often means the standby region's readiness is untested until the day you actually need it. Active-active serves traffic from both regions simultaneously, giving faster failover and validating both regions continuously, at the cost of real complexity around data consistency and conflict resolution. For most pods I'd default to active-passive unless their SLO genuinely can't tolerate failover time — active-active is a meaningful engineering investment that should be justified by the SLO, not adopted by default.

CHAO-06 Give a concrete chaos experiment example with an expected outcome.

Hypothesis: "checkout stays under our 300ms p99 latency SLO even if one of three backend pods is killed." Experiment: LitmusChaos pod-delete targeting one replica of the checkout deployment, 5-minute observation window, abort automatically if error rate exceeds 5%. Expected outcome if healthy: Kubernetes reschedules the pod, the service mesh/load balancer routes around the gap, and golden-signal dashboards show a brief blip within the SLO's error budget — not a customer-visible outage. If it doesn't hold, that's the actual finding: maybe readiness probes aren't fast enough, or there aren't enough replicas for one loss to be absorbed cleanly.

CHAO-07 A pod lead is skeptical about running their first chaos experiment in production. How do you bring them along?

I wouldn't start in production at all — start in staging, with a scenario they already believe should be safe, so the first experience is confidence-building rather than risky. Show the blast-radius controls and abort conditions concretely before asking for buy-in, and offer to run the first one myself, live, with them watching, rather than asking them to run something unfamiliar on their own service. Trust builds from a small, successful first experiment — not from a persuasive pitch about chaos engineering's value in the abstract.

CHAO-08 A DR test reveals the actual RTO is 3x the documented target. What's the guild-level follow-through?

First, get the honest gap in front of the pod lead and delivery leadership immediately — a DR gap discovered in a controlled test is exactly the win the test was meant to produce, not a failure to hide. Then work with the pod on a remediation plan with a real deadline, and re-test against that deadline rather than letting the finding fade into a backlog item nobody circles back to. I'd also check whether the same gap pattern exists in other pods with a similar architecture — one bad DR test is often not actually isolated to just that one pod.

Guild Operations — Runbooks, Metrics & Career Pathing

The operational machinery behind guild leadership: maintaining shared runbooks/playbooks, mentoring, contributing to hiring, and reporting guild health — all explicitly named in the JD's responsibilities.

GUIL-01 How would you structure a shared runbook library so it's actually usable across pods during an incident?

Indexed primarily by **symptom** (what an on-call engineer is actually seeing at 2am — "high error rate on X," "pods CrashLoopBackOff") rather than purely by service, since during an incident you often don't yet know which service is truly at fault, just what you're observing. Each pod's service-specific runbooks live alongside guild-wide cross-cutting ones (network issues, shared dependency outages), with a consistent template so an engineer from Pod A can follow Pod B's runbook without a learning curve during a cross-pod incident.

GUIL-02 How do you keep runbooks from rotting?

Explicit ownership per runbook (usually the pod that owns the service), a review trigger every time it's actually used in an incident (did it work as written — update immediately if not, while it's fresh), and a quarterly guild-level sweep flagging anything untouched for two quarters as a review candidate. A runbook without a named owner is the single most reliable way to guarantee it goes stale.

GUIL-03 Which 4–5 guild health metrics would you actually put in front of leadership, and why those?

SLO attainment rollup, MTTR trend, incident volume/severity mix, automation/toil-reduction coverage, and standard-adoption rate across pods. I'd deliberately keep it to a handful — a guild health report leadership actually reads monthly beats an exhaustive one they skim once and ignore. Each metric should map to a decision leadership might need to make (where to invest, which pod needs help), not just be measured because it's measurable.

GUIL-04 How do you run knowledge-sharing sessions across pods that people actually show up to?

Short (30 minutes), practical, and tied to something a pod just lived through — a recent incident's lesson, a tool that saved real time, a mistake worth everyone avoiding — rather than abstract "best practices" talks. Rotating the presenter across pods (not always me) builds ownership and surfaces expertise the guild might not otherwise know exists in a pod. I'd record sessions for the offshore/different-timezone folks who couldn't attend live, given TCG's onshore/offshore delivery model.

GUIL-05 What would you look for in an SRE candidate specifically for TCG Digital's pod-embedded reality?

Beyond the technical bar: comfort with ambiguity and context-switching (a pod-embedded SRE may work across different client stacks over their tenure, unlike a single-product company's SRE), some client-facing communication comfort since pods work directly with client engagements, and — importantly — genuine collaborative instinct rather than a pure command-and-control operational style, since success here means influencing a pod, not commanding one. I'd weight "can they explain a technical tradeoff to a non-technical stakeholder" more heavily here than at a product company, given how client-facing the pod model is.

GUIL-06 What's the on-ramp for an engineer inside a pod who wants to grow into SRE work?

Start them on guild-adjacent work inside their own pod first — owning their service's SLO, writing their first runbook, taking rotation on-call — before pulling them into guild-wide activities like presenting at a sync or contributing to a shared module. I'd pair them with the guild's competency rubric (from the hiring/career-pathing question above) so growth has a visible ladder, and invite them into guild syncs as an observer early, since exposure to cross-pod thinking is itself part of the development.

GUIL-07 Sketch a 30/60/90 for onboarding a brand-new pod onto the guild's standards.

- **30 days** — baseline audit (what do they have today), golden-path chart adopted for at least one service, first SLO defined
- **60 days** — on-call/escalation path integrated with guild-wide routing, first guild sync attendance, runbook template in use
- **90 days** — full golden-signal dashboard coverage, error budget policy agreed with pod lead, first postmortem run through the guild's blameless format

I'd keep this flexible rather than a rigid checklist — a pod under a hard client deadline in month one needs a different pace than one with breathing room, and forcing the same timeline on both is how onboarding becomes box-checking instead of real adoption.

GUIL-08 How do you evaluate whether to introduce a new tool guild-wide, without causing tool sprawl?

A lightweight pilot framework: one pod trials it for a fixed period against a real problem, report back what it actually solved versus what it cost to adopt, and only then propose guild-wide rollout — never adopt guild-wide off a vendor demo or a conference talk alone. I'd also require a genuine gap to justify a new tool ("we don't have anything for X") rather than a marginal improvement over something the guild already standardized on, since every additional tool is ongoing maintenance and training cost multiplied across every pod that adopts it.

Cross-Guild Collaboration

The JD names two specific peer guilds: DevOps & Infra, and Architecture & Design. Knowing where your guild's boundary sits relative to each is a strong, specific signal.

CROS-01 Concretely, what does the SRE guild bring to a solution design review with the Architecture & Design guild?

Non-functional requirements grounded in operational reality: what does this design cost to run reliably (SLO feasibility given the proposed architecture), what's the failure mode under load, what does on-call actually look like for this once it's live, and does it fit the guild's observability/deployment golden paths or require a bespoke exception. I'd aim to be in the room during design, not just reviewing after the fact — NFR feedback after a design is finalized is far more expensive to act on than the same feedback during design.

CROS-02 Where does the SRE guild end and the DevOps & Infra guild begin? How do you avoid turf conflict?

A reasonable split: DevOps & Infra guild owns the underlying platform — cluster provisioning, networking, the CI/CD tooling itself. SRE guild owns the practice layered on top — SLOs, incident process, observability standards, the golden-path templates that *use* that platform. In practice the boundary gets fuzzy (is a Helm chart's default resource limits an SRE guild standard or a DevOps & Infra platform default?) — I'd resolve that with a standing sync between the two guild leads rather than a rigid RACI nobody actually follows, and treat any recurring ambiguity as a sign the two guilds should just co-own that specific artifact.

CROS-03 How do you frame a capacity or staffing gap to delivery leadership so it actually lands?

Business impact language, not "we're short-staffed": "at current pace, Pod X's on-call burden risks burnout by [timeframe], and that's a retention and incident-response-quality risk for a client-facing service" lands very differently than "we need more people." I'd back it with the guild health data already being tracked (incident volume trend, MTTR, alert-to-page ratio) rather than an anecdotal ask — leadership can act on a trend line far more easily than on a feeling.

CROS-04 The Architecture & Design guild proposes a design you think is operationally risky. How do you push back constructively pre-launch?

Bring the specific risk with a concrete scenario, not a vague "this feels risky" — "under a 2x traffic spike, this synchronous call chain has no circuit breaker, and a single slow dependency takes down three services with it." Offer an alternative or mitigation, not just an objection, and pick your battles: if the risk is real but the mitigation is expensive relative to actual likelihood, say so honestly rather than blocking on principle. Guild credibility depends on pushback being proportionate and specific, not reflexive.

CROS-05 Architecture guild wants speed to market; SRE guild wants a reliability gate. How do you resolve the conflict?

This is rarely actually a binary — I'd look for the smallest reliability gate that catches the highest-risk failure modes without materially slowing the timeline (e.g., "ship without the full chaos-tested DR plan, but not without basic health checks and a rollback path"), and make the tradeoff explicit and owned by delivery leadership if it can't be resolved technically — my job is to make the risk visible and quantified, not to unilaterally block a business decision that isn't mine alone to make.

Scenario & Troubleshooting Bank

Cross-pod, guild-flavored scenarios — the kind of question that tests judgment under the specific constraints of this role, not just technical recall.

SCEN-01 Multiple pods independently report the same intermittent latency spike, traced to a shared platform service. Walk through your response.

First move: stop treating it as N separate pod issues — open one shared incident channel spanning affected pods immediately, since duplicate parallel investigations waste time and can even conflict (two pods independently "fixing" the same shared dependency). I take or assign incident command since I'm the only one with visibility across all affected pods. Root cause ownership goes to whoever owns the shared service (likely DevOps & Infra guild if it's platform infrastructure); each pod reports impact and applies their own local mitigation (retries, circuit breakers) in parallel while the root cause gets fixed upstream. Postmortem afterward is guild-level, not pod-level, since the fix needs to prevent recurrence for every pod depending on that shared service, not just the ones that happened to notice first.

SCEN-02 A pod ships a change mid-quarter that blows through their error budget. What do you do?

Check whether the pod's pre-agreed error budget policy actually triggers here (it should, if it's a real policy) — feature work pauses, reliability work takes priority until budget recovers. I'd have that conversation with the pod lead directly and quickly, not let it linger un-enforced, because an error budget policy that's ignored the first time it actually matters loses all credibility for every pod watching how it plays out. I'd also dig into whether this was a one-off bad release or a symptom of a pattern (rushed testing, no canary) worth a guild-level process fix.

SCEN-03 You're asked to be the technical judge between two pods proposing different logging solutions. How do you decide?

I'd anchor the decision in the guild's actual standard (does one option satisfy the golden-path observability contract better, does one already have wider adoption that makes consolidation cheaper) rather than a pure technical merits debate — the "best" logging tool in isolation matters less than which choice reduces guild-wide fragmentation. If both are genuinely equivalent against the standard, I'd let the pods' own preference and existing familiarity decide rather than imposing my personal pick — forcing a choice I don't have to live with day-to-day is exactly the over-reach that erodes trust in guild judgment.

SCEN-04 **A critical incident hits a pod running a stack you don't know well. Walk through your first 15 minutes.**

Same discipline as the incident-management section: I don't try to become the domain expert — I confirm someone's driving the fix and someone's scribing, ask the cross-cutting questions the pod might be too close to ask ("what changed recently," "seen this pattern elsewhere"), track SLO burn rate against a mitigation decision point, and protect the pod from stakeholder noise so they can stay heads-down. My unfamiliarity with their specific stack is exactly why I stay in the coordination role rather than trying to drive the technical fix myself.

SCEN-05 **A guild standard proposal gets rejected by 3 of 5 pod leads. What's your next move?**

That's a real signal, not just resistance to push through — I'd go back to the three who rejected it and actually understand why, since "no" from multiple independent pods usually means the standard has a genuine gap, not that three teams are simultaneously being difficult. I'd revise based on that feedback and re-propose, or if the underlying need is proven by the two pods who did want it, offer it as opt-in for now rather than forcing consensus that isn't there. A guild lead who pushes a rejected standard through anyway on authority they don't actually have burns credibility fast.

SCEN-06 **A cost spike on shared Kubernetes infrastructure traces back to one pod's misconfigured autoscaling. How do you handle it?**

Immediate technical fix first — cap the HPA/cluster-autoscaler bounds to stop the bleeding — then the conversation with the pod, not the other way around; a cost conversation while the meter's still running gets tense for no reason. Root cause is usually a missing or wrong maxReplicas / resource request, so beyond the immediate fix, I'd add this as a guild-wide lint check (flag any HPA without an explicit max) so the same misconfiguration can't recur silently in another pod.

SCEN-07 **A DR test reveals RTO is 3x the target. How do you drive the fix without owning the service?**

Same as the chaos-engineering answer: surface the honest gap immediately to the pod lead and delivery leadership, agree a remediation plan with a real deadline together with the pod (I don't own their service, so I can't unilaterally assign myself the fix), and re-test against that deadline rather than letting the finding quietly age into an ignored backlog item. I'd also check whether the gap pattern shows up in other pods with similar architecture, since it rarely is truly isolated.

SCEN-08 **A new client engagement stands up a pod overnight with an aggressive deadline. How do you get baseline reliability practices in without slowing them down?**

I wouldn't hand them the full standards checklist on day one — that's how a guild becomes the thing that slows down a pod under real deadline pressure, which is exactly the reputation you can't afford. I'd offer the smallest viable slice: the golden-path chart (gets them dashboards/alerts basically for free), a lightweight first SLO, and a point of contact for questions — full guild onboarding (runbooks, guild sync attendance, postmortem process) comes once they're past the initial crunch. Meeting a pod where they actually are, under real constraints, is what makes "influence without authority" credible instead of theoretical.

Behavioral, Positioning & Closing

CLOS-01 Why TCG Digital, specifically?

Ground this in something real rather than a generic "great culture" answer. TCG Digital is the AI and digital arm of a genuinely diversified, multi-billion-dollar conglomerate — that scale and stability is worth naming if it matters to you at this career stage. Their pod/Factory delivery model means real variety in what you'd touch (their mcube platform won a national AI-product award in 2025, and public messaging leans hard into agentic AI, including in regulated GxP/pharma environments) — which is a genuinely interesting reliability problem: standards that hold up across different client domains and compliance bars, not just one product.

If you want a second layer: this guild-lead structure is a chance to operate at a level of technical leadership breadth that a single-product SRE role wouldn't offer — cross-pod influence, not just one team's reliability.

CLOS-02 Why a guild-lead role specifically, rather than a pure IC SRE role, at this point in your career?

With 14+ years, you've almost certainly already been doing informal guild-lead work inside teams — setting technical direction other engineers followed without a formal reporting line, being the person newer engineers go to. This role is a chance to make that the explicit job instead of a side effect of being the senior person in the room, without taking on the parts of people-management (headcount, performance reviews) that pull time away from staying technically hands-on — which, for someone with your depth, is often the more attractive tradeoff than a traditional EM path.

CLOS-03 The JD says AWS, Azure, or GCP — how do you position your Azure certifications against your AWS hands-on experience?

This JD is actually one of the more favorable ones for this — it explicitly accepts any of the three, not AWS-only. Lead with the real hands-on breadth (your self-assessed 6+ years AWS, 6+ years Kubernetes, 3–6 years Terraform), then mention the AZ-104/SC-300/SC-100 certifications as evidence you're comfortable operating across cloud providers, which is directly relevant to a guild that needs to set standards pods running on *different* clouds can all actually follow. Frame the multi-cloud fluency as an asset for this specific role's governance angle, not as a gap to explain away.

CLOS-04 If they probe on architecture ownership — "did you design this, or did a platform team?" — how do you answer honestly without undercutting yourself?

Don't say "I didn't do that" flat — reframe accurately: "the platform/architecture team owned the initial design; my role was operating, troubleshooting, and evolving it in production" is both more honest and a stronger answer, since production ownership and evolution is real, valuable depth even when you didn't originate the design. For a guild-lead role specifically, this framing is doubly relevant — the role explicitly isn't about being the sole architect, it's about setting standards and improving what's already running across many pods' systems you didn't personally design.

CLOS-05 Tell me about a time you mentored someone.

Pick a real example — ideally one where the person you mentored ended up capable of something without you, not just "I helped them once." The guild angle to bring out if it fits your real story: mentoring someone who didn't report to you is a much stronger signal for this role than mentoring a direct report, since it's the exact muscle the guild-lead job needs. Structure it STAR-style and keep it concrete — what specifically did you teach, how did you know it landed.

CLOS-06 Tell me about your biggest production incident and what changed afterward.

Pick a real incident from your 14+ years and be honest about what went wrong — interviewers are usually more interested in what changed systemically afterward than in the drama of the incident itself. The strongest version of this answer names a concrete process or tooling change that came out of the postmortem, ideally one that's still in place, showing the learning stuck rather than the incident just being survived and forgotten.

> *Tip: **Prep note:** pick your real story tonight and rehearse the specific "what changed" part — that's the part interviewers actually probe on follow-up, and it's the part people tend to under-prepare.*

CLOS-07 How do you stay current in a field that moves as fast as SRE tooling does?

Name your actual habits honestly — following specific projects' release notes, hands-on experimentation before adopting anything guild-wide, engaging with the SRE/platform-engineering community. For this role specifically, connect it back to the guild's "stay current on SRE tooling, evaluate and introduce new technologies" responsibility — the pilot-first framework from the guild-operations section is a good concrete answer to "how would you evaluate whether something new is actually worth guild-wide adoption," rather than just "I read a lot."

CLOS-08 Notice period and compensation expectations — how do you handle these if asked?

Have your actual notice period ready as a plain number rather than hedging (30 days is standard for most India-based roles at your level, but confirm your own contractual obligation). For compensation, it's fine to give a range grounded in your research of comparable AVP/Guild-Lead-equivalent SRE roles rather than a single hard number this early, and it's reasonable to ask about their band before committing to specifics if they ask first — this is a negotiation, not a test with one right answer.

CLOS-09 What should you ask the interviewer at the end?

- "How many pods currently fall under the SRE guild's scope, and how varied are their tech stacks and client domains today?"
- "Where is the guild model today — is this a brand-new function, or am I inheriting existing standards and adoption?"
- "How is guild success actually measured internally right now, if at all?"
- "What's the single biggest reliability pain point across pods today that made this role a priority to fill?"
- "How does the SRE guild typically interact day-to-day with the DevOps & Infra and Architecture & Design guilds — is that relationship already well-defined?"
- "Given the recent push into agentic AI, is the guild expected to extend reliability practice into AI/ML-served workloads, or is that scoped separately for now?"

These do double duty — they're genuinely useful things to know, and asking them signals you're already thinking like someone operating the role, not just interviewing for it.

Round 2

Greenfield, Leadership & Fit · 7 sections · 58 questions

Greenfield — Building the SRE Practice From Zero

TCG Digital confirmed this is a greenfield guild — no existing SRE practice to inherit. That shifts the interview's center of gravity here: expect this to be the most heavily probed territory in round two, especially the 90-day question below.

ZERO-01 [KEY] If we hire you as Guild Lead, what will you do in your first 90 days?

This is very likely the single most important question in the whole process, precisely because they've told you it's greenfield — there's no existing guild to describe improving, so this question effectively *is* the interview for a founding-level hire. The answer needs to show a deliberate sequence, not a list of good intentions in random order.

Days 1–30 — Assessment. Nothing gets mandated yet.

– Meet every pod lead one-on-one, not just the loudest or largest pods
– Inventory what already exists per pod — any monitoring, any on-call, any IaC, any documented incident history
– Rank pods by risk — client-facing exposure, incident history, blast radius if something breaks
– Deliverable: a one-page current-state map and risk ranking, shared back with delivery leadership as the "here's what I actually found" moment — proof you listened before you proposed anything

Days 31–60 — Standards, first SLOs, first observability baseline — piloted, not mandated.

– Draft a light, one-page guild charter based on what was learned in month one, and socialize it with pod leads before calling it final
– Pick one willing, real-pain pod as the pilot — get them a first SLO defined and a golden-path observability template applied (the four golden signals, wired up, not theoretical)
– Publish a short, defensible "non-negotiables" list versus "recommended" patterns — small enough to actually be enforced, not a wishlist

Days 61–90 — Incident process, toil reduction, roadmap, adoption, and the first metrics.

– Formalize severity classification and escalation — at minimum for the pilot pod, in outline for the rest — with the guild lead as the senior escalation point
– Find the single highest-leverage toil pattern shared across multiple pods and ship one real automation or self-service fix for it
– Publish a sequenced two-to-three-quarter reliability roadmap, tied directly to the risk ranking from day one — not a wishlist
– Bring a second and third pod into the fold — adoption expanding past the single pilot
– Deliver the first guild health report to leadership — SLO attainment, incident trend, adoption percentage — even if the numbers are still rough, because a founding guild's first job is proving it produces something measurable

Say the sequence out loud if asked to summarize it: **assessment -> standards -> SLOs -> observability -> incident process -> toil reduction -> reliability roadmap -> guild adoption -> measurable outcomes.** That exact order matters — standards before assessment reads as arrogance, automation before there's an SLO to protect it reads as busywork, and adoption claimed before it's actually earned reads as spin. The interviewer isn't just checking whether you know these ingredients; they're checking whether you'd sequence them correctly under real greenfield pressure.

*Tip: **Prep note:** rehearse this one out loud, not just in your head — it's long enough that the first time you say the full sequence shouldn't be in the interview room.*

ZERO-02 How would you establish an SRE practice from zero, if none exists today?

The same principles as the 90-day plan, stated more generally: start with the highest-leverage, lowest-ceremony win rather than the theoretically most important one. Earn adoption pod by pod instead of mandating guild-wide from day one. Make the golden path genuinely easier than the status quo, so adoption is the path of least resistance rather than an ask. And measure before you claim improvement — you need a real baseline before "better" means anything.

ZERO-03 What's the very first deliverable you'd ship, before anything else, to start earning trust?

Not a charter, not a mandate — something genuinely useful, fast. Helping one pod through their next real incident, shipping them a working dashboard, or writing their first runbook *with* them rather than handing them one. In a greenfield role, trust comes from a concrete, useful artifact landing in someone's hands within the first couple of weeks — a strategy document alone doesn't earn anything yet.

ZERO-04 How do you decide which standard to establish first, when there's no existing practice to build on?

Start where the pain is worst and most visible — the pod with the highest incident volume or severity — and where the fix is genuinely achievable in weeks, not months. The first standard should be small enough to actually land and prove the model works, not the most theoretically complete one. A greenfield guild that opens with its most ambitious standard usually fails to land any of them.

ZERO-05 How would you design a production-support strategy from scratch, before there's a guild at all?

Start by defining the boundary that doesn't exist yet: what counts as routine production support (a ticket, handled within business hours) versus a true incident (a page, handled now). Early on, the pod itself is the first line of support — the guild's role is advisor and escalation resource, not first responder, since guild-wide on-call infrastructure doesn't exist yet. I'd design the escalation path to *scale* as more pods join, rather than building it as if guild-wide coverage already exists on day one.

ZERO-06 How does an SRE guild relate to platform engineering — are they the same thing?

Not quite the same: platform engineering builds the paved road — the tooling, the self-service platform, the golden-path infrastructure itself. The SRE guild defines the practice and standards that road needs to enforce — SLOs, incident process, the reliability bar. In a greenfield org without a dedicated platform team yet, a guild lead often ends up doing a bit of both early on — building the first golden path *and* defining the practice around it — which is worth naming honestly in this interview rather than pretending the two are already cleanly separated from day one.

ZERO-07 How do you measure success for a guild starting from zero, with no baseline to compare against?

The first 90 days measures **existence and adoption** — is there now a charter, does one pod have an SLO that didn't exist before, did a first incident actually go through a real process — not **improvement**, which needs a baseline that doesn't exist yet. Once two or three quarters of real data accumulate, the metrics shift to trend — MTTR improving, incident volume dropping. Trying to show improvement in month one is measuring the wrong thing entirely, and a candidate who claims early improvement numbers in a genuinely greenfield context should raise your own skepticism, not just the interviewer's.

ZERO-08 How do you identify toil in an environment where nothing has been measured before?

Ask pod engineers directly what they hate doing repeatedly — this is often faster and more accurate early on than trying to instrument toil measurement from a standing start. Shadow an on-call rotation or two to see it firsthand. And watch for the same complaint surfacing independently across multiple pods — that's the highest-leverage guild-wide target, since it justifies guild-level investment rather than a one-pod local fix.

ZERO-09 With no existing cloud or Kubernetes foundation to inherit, how do you approach the platform layer's architecture as guild lead?

Not as the person personally architecting the cluster from scratch — that's more likely the DevOps & Infra guild's job, or whichever team owns the platform build. As guild lead, the job is making sure reliability requirements — a multi-AZ minimum, standard observability hooks, a golden-path Helm baseline — are designed into that foundation from day one, since retrofitting reliability into infrastructure after the fact is far more expensive than building it in. I'd want a seat at that architecture conversation early, even if I'm not the one holding the pen.

ZERO-10 What's the single biggest risk in a greenfield guild build, and how do you avoid it?

Over-engineering process before there's any adoption pull — arriving with a lengthy standards document and a mandatory checklist before a single pod has volunteered to try anything. That's what kills a greenfield guild's credibility fastest, and it's an easy trap for a senior hire eager to show up with a complete plan on day one. The mitigation is the thread running through every answer in this section: earn the first real adoption with something genuinely useful, then scale from proof, not from decree.

Executive & Strategic Leadership

A likely second-round shift: from "can you do the job" to "can we trust you at a higher altitude" — budget, tradeoffs across the whole Dev Factory, and a multi-year view of the guild.

EXEC-01 How would you present a reliability investment case to VP-level delivery leadership?

Translate reliability into numbers leadership already tracks: incident hours × engineer cost = the cost of unreliability; a client-facing SLA breach risk = revenue and relationship risk; toil hours reclaimed = capacity redirected to billable or feature work. Framed as "cost avoided" and "capacity unlocked," not as an abstract engineering-quality ask — that's the language that actually gets budget approved.

EXEC-02 Two pods both need deep guild attention this quarter, but you can only really support one. How do you decide, and how do you communicate that upward?

Objective criteria, not gut feel: client-facing risk severity, blast radius if the current gap causes a failure, and leverage — sometimes a small nudge gets a nearly-there pod fully compliant faster than a huge lift moves a far-behind pod partway. Whichever way I decide, I say so explicitly to delivery leadership rather than quietly choosing — an invisible resourcing tradeoff nobody signed off on is how a guild lead loses trust the first time the "wrong" pod finds out later.

EXEC-03 Where do you see the SRE guild at TCG Digital in two to three years?

From "guild lead evangelizing standards pod by pod" to "the golden path is just how a new service gets built" — most guild interaction becomes proactive (a design review, a mentoring conversation) rather than reactive (chasing a gap). Ideally, guild health metrics are baked into how new client engagements get scoped from day one, not bolted on after a pod's already live.

EXEC-04 How does this guild-lead role build toward a broader platform or AVP-level role later?

The honest answer: it largely writes its own case, if done well. Concrete, org-wide outcomes — adoption rate, MTTR trend, incident reduction — done at guild scale across many pods is close to the actual evidence an AVP-level platform-leadership case needs. I'd track those numbers deliberately from day one, not scramble to reconstruct them at promotion-conversation time.

EXEC-05 A VP asks you to cut reliability investment to hit a cost target this quarter. How do you respond?

Not a flat refusal — a clear translation of what's actually being traded: which pods or SLOs this touches, and what risk goes up as a result. I'd give the VP a real, informed choice, and if a cut has to happen, argue for cutting the lowest-risk item first rather than an across-the-board reduction that damages everything a little. My job is making the tradeoff visible, not overriding a legitimate business call that isn't mine alone to make.

EXEC-06 How do you measure and report the guild's ROI in terms leadership actually cares about?

Incident-hours avoided (estimated from the trend before and after a fix), toil-hours reclaimed and redirected to feature work, and reduced client escalation frequency — each translated into cost or client-relationship terms, not left as raw engineering metrics leadership has to interpret themselves.

EXEC-07 How would you handle a strategic disagreement with the Head of Engineering or Delivery about the guild's scope?

Lead with data — adoption trend, incident trend — not opinion. Assume there's a constraint I don't see (revenue pressure, a client commitment) rather than assuming they're simply wrong. Propose a smaller pilot instead of an all-or-nothing ask, and know when to accept their call gracefully once genuinely heard — a guild lead has real influence, not veto power over delivery leadership's actual authority, and pretending otherwise burns credibility fast.

EXEC-08 What's your framework for deciding which reliability initiatives deserve guild-wide investment versus staying pod-local?

Guild-wide if the pattern shows up across three or more pods, if it's foundational (on-call routing, golden-path templates) rather than service-specific, or if the risk sits at the level of client trust rather than one service's uptime. Otherwise it stays local — guild-wide investment in a genuinely one-off problem is waste, and pods notice when the guild over-invests in things that don't actually generalize.

EXEC-09 How do you build a business case for headcount you don't own, when the guild is stretched thin?

Data first — adoption requests outpacing guild capacity, response times degrading — framed as delivery leadership's decision to make with real evidence in front of them, not a bare "I need more people." I'd also show what I've already tried (embedded champions, automation investment) so the ask reads as a considered escalation, not a first resort.

Client-Facing & Business Acumen

TCG Digital's pods sit directly under client engagements — a second round is a plausible place to test whether you can operate reliability practice in a services context, not just a product one.

CLIE-01 A client stakeholder pushes back on an SLO as "too conservative" and wants a tighter number for a sales pitch. How do you handle it?

I don't let a sales pitch override engineering reality — I'd show the data behind the current number, offer to genuinely explore what a tighter SLO would cost or require, and let account and delivery leadership make the final call on that risk with the real tradeoff in front of them, rather than quietly agreeing to a number engineering can't actually hit.

CLIE-02 Walk through presenting a reliability roadmap or postmortem summary to a non-technical client stakeholder.

Lead with business impact — what happened, who was affected, for how long — and skip the technical root-cause detail unless they specifically ask for it. The client wants confidence and a credible forward plan, not a play-by-play of the technical fix; over-explaining the technical side can actually undermine confidence rather than build it.

CLIE-03 A pod's delivery lead wants to skip a guild reliability gate to hit a client deadline. How do you navigate that?

First, understand what's genuinely at risk if it's skipped — not every gate carries the same weight. I'd offer the fastest *safe* path rather than a flat no, and if the risk is real, make sure delivery leadership makes that call explicitly rather than the standard getting quietly overridden under deadline pressure with nobody accountable for the decision.

CLIE-04 How would you contribute technical credibility to an RFP or proposal for a new client engagement?

Help scope realistic reliability commitments before they're promised to a client — the worst outcome is sales promising an SLA engineering can't actually hit. I'd also position the guild's standards themselves as a genuine differentiator in the pitch: "here's how we'll operate this reliably from day one" is a real sales asset for a services company, not just an internal engineering nicety.

CLIE-05 A client asks "why should we trust your reliability practice" in a QBR. What's your answer?

Specifics, not reassurance — SLO attainment data, the incident response process, the blameless postmortem discipline behind it. "We take reliability seriously" convinces nobody in that room; a concrete, measurable practice does.

CLIE-06 A client's own technical requirements conflict with your guild's standard. How do you handle it?

Understand the client's real constraint first — sometimes it's a genuine contractual or compliance requirement, sometimes just unfamiliarity with why the guild standard exists. Where possible, I'd adapt the implementation while holding the underlying outcome constant — the same "standardize the contract, not the tool" instinct from pod-level guild work, just applied to a client relationship instead.

CLIE-07 A client-facing incident becomes a trust issue, not just a technical one. How do you approach it differently?

Treat the technical fix and the relationship repair as two separate tracks needing separate attention. I'd overcommunicate on the relationship side — real transparency, a credible prevention plan, possibly a direct call from delivery leadership — even once the technical issue is resolved, since trust doesn't recover on the incident's timeline; it recovers on its own, slower one.

CLIE-08 **How do you translate a technical risk — tech debt, a single point of failure — into language a client executive will actually act on?**

Business risk with a rough likelihood and cost if it's realized: "if this fails during your peak season, here's the customer impact and the revenue exposure." Executives act on risk to their business, not on an engineering purity argument about code quality for its own sake.

CLIE-09 **Multiple client engagements report reliability concerns in the same quarter. How do you prioritize fairly across clients?**

The same objective criteria as prioritizing across pods internally — severity and blast radius, contractual SLA exposure, and how close each is to resolution with modest effort — and I'd be transparent with delivery and account leadership about that logic, so no client relationship feels arbitrarily deprioritized behind closed doors.

Deeper Behavioral & Leadership Philosophy

A second round often re-asks behavioral territory with sharper follow-ups. These are deliberately different angles from round one's behavioral set — pull real examples, and where a question is inherently personal, use the framing as a structure to fill in, not a script.

PHIL-01 **Tell me about a time you were wrong about a significant technical or strategic call.**

Use STAR, and put the weight on how you *found out* you were wrong and what you did next — not on defending the original call. Interviewers at this stage are listening for genuine self-correction, not a well-disguised justification.

> *Tip: **Prep note:** pick a real one from your 14+ years tonight — this question rewards specificity far more than a smooth, generic answer.*

PHIL-02 **A senior developer on a pod pushes back that your reliability gate slows down their releases and isn't worth the friction. How do you handle that disagreement?**

This is different from a pod lead declining on time grounds — it's an individual engineer disputing the gate's value on principle, and it deserves a technical answer, not an authority one. I'd ask them to walk me through a specific release the gate actually slowed down, and look honestly at whether the friction was proportionate to the risk it caught — sometimes they're right and the gate is miscalibrated for that service. If the gate is genuinely justified, I'd show them the failure mode it exists to catch, concretely, rather than asserting the standard's importance. Losing that argument gracefully when they have a point is part of staying credible with engineers who don't report to you; so is holding the line, clearly and respectfully, when they don't.

PHIL-03 **How do you give hard feedback to a senior engineer in a pod who doesn't report to you?**

Direct and private, focused on specific behavior and its impact rather than character, framed around a shared goal — the standard, the incident, the client outcome — rather than personal authority. With no positional leverage behind it, the feedback has to be persuasive entirely on its own merits, which usually makes it better feedback, not worse.

PHIL-04 **Describe a time you disagreed with someone more senior than you.**

STAR again — the interesting part is *how* you raised it (data and reasoning, not just a differing opinion) and what actually happened, including if you turned out to be wrong. A candidate who was right every single time they pushed back reads as either unbelievable or not self-aware — a mixed record, honestly told, lands better.

PHIL-05 **What's the biggest professional regret in your career, and what changed because of it?**

Pick something real and specific. The weight of the answer should sit on the concrete change in how you operate now — not on the mistake itself or an extended apology for it.

PHIL-06 **How do you know when to push harder on a standard versus letting it go?**

Weigh actual risk against relationship cost. Push hard on genuine non-negotiables — safety, data integrity, a real client commitment. Flex on style or tooling preference where the outcome is equivalent either way. Treating every standard as equally non-negotiable is how a guild lead runs out of political capital fast.

PHIL-07 **What's your leadership philosophy, broadly — not the guild-specific version from round one?**

This one's genuinely yours to answer, not something I should hand you pre-written — a philosophy that sounds coached rather than lived tends to show. Spend a few minutes tonight putting your real one into a sentence or two, grounded in how you've actually operated across your career, and say it plainly rather than reaching for something that sounds impressive.

PHIL-08 **Tell me about a time your reliability recommendation was overridden. What happened, and were you right?**

STAR, and be honest if the answer is that you were wrong, or partly wrong — an interviewer will trust a candidate who admits a mixed track record far more than one who claims to have always been right and simply ignored.

PHIL-09 **How do you handle burnout risk in yourself, given the scope of a guild-wide, cross-pod role?**

Name real, credible habits — boundaries around being the default escalation point at all hours, actually delegating through embedded pod champions rather than staying the single point of contact, protecting focused time instead of living entirely in reactive mode. This isn't a soft wellness question in a guild-lead interview — a guild lead who burns out stops being useful to every pod depending on them, so it's a legitimate operational question about the role's sustainability.

PHIL-10 **What does "senior" actually mean to you, at this stage of your career?**

Worth answering in your own words, grounded in your real 14+ years rather than a textbook definition — things like judgment under ambiguity, multiplying other people's effectiveness rather than just your own output, and being right more often than wrong on calls nobody's double-checking are the usual shape of a strong answer, but make it genuinely yours.

Culture, Values & Fit

Grounded in TCG Digital's own public framing — a genuinely multicultural, multi-country delivery org running on a services model, with "Velocity to Value" as its stated mantra.

CULT-01 **TCG's culture leans hard into diversity and global teams. How do you operate in a genuinely multicultural, multi-timezone team?**

Overcommunicate in writing so nothing important depends on being in the same room or time zone, and be deliberate about not letting whoever's loudest or in the most convenient timezone dominate guild syncs. I'd treat communication-style and cultural differences as something the guild actively accounts for in how it runs meetings and documents decisions, not something people are just expected to adapt to unassisted.

CULT-02 **"Velocity to Value" is TCG's mantra. How does the speed-versus-rigor tension show up in guild work, and how do you navigate it?**

The guild's job isn't to slow velocity down — it's to make the fast path also the safe path, which is exactly what a golden-path template is for. Where genuine tension remains, I'd make the tradeoff visible and let the business decide with eyes open, rather than the guild either unilaterally blocking speed or silently absorbing risk it never surfaced.

CULT-03 **This is a services and consulting culture — delivery means adapting to a client's priorities, not just your own roadmap. How do you talk about that shift?**

Honestly: a guild lead in a services company has to hold real standards while staying genuinely flexible on timing and sequencing around client reality. Rigidity that ignores client context — insisting on the "right" way regardless of what a client engagement actually needs right now — is exactly how a guild loses credibility fast in this kind of organization.

CULT-04 **How do you stay motivated in a role where success depends on other people's voluntary buy-in, not direct control?**

Answer this with something genuinely true for you — visible impact when a pod's incident count actually drops because of something you introduced, the intrinsic pull of influence-based leadership itself, or the variety of touching many different pods' problems rather than one team's. A generic "I love collaboration" line won't land as well as something specific and real.

CULT-05 **What would make you walk away from an offer, even a good one?**

Worth having a genuine, specific answer ready — values misalignment, a scope that doesn't match the stated authority, a compensation floor — rather than a diplomatic non-answer. Interviewers generally respect a real boundary more than a vague deflection here.

CULT-06 **How do you define success for yourself in your first year in this role?**

Tie it to something measurable and guild-appropriate: a real number of pods genuinely following the standard (not just told to), a measurable drop in incident recurrence, and credibility earned with at least a handful of pod leads who'd vouch for the guild's value without being asked to.

CULT-07 **Where do you want to be in five years, and how does this role fit?**

Connect this honestly to your own trajectory toward broader platform or architecture-level leadership — the guild-lead scope here is a natural stepping stone toward that, worth saying plainly if it's genuinely true for you rather than dodging the question with vague "I want to keep growing" language.

HR, Compensation & Logistics

The practical conversation that usually shows up once a company is genuinely interested — worth walking in with real answers ready rather than improvising.

HR-01 **How do you handle "current CTC / expected CTC" without underselling yourself?**

State current CTC factually if asked directly, and give expected as a realistic range grounded in actual market data for AVP or guild-lead-equivalent senior SRE roles — not a single number, and not anchored low just to seem easy to work with. Negotiate from research, not guesswork.

HR-02 What's a reasonable notice period to state, and how do you handle a company wanting you sooner?

State your actual contractual notice honestly rather than what you think they want to hear — commonly 30–90 days depending on your current employer and level in India, so confirm your own real obligation before the conversation. If they push for sooner, that's a negotiation between them and your current employer's terms, not something you should promise a date for on the spot.

HR-03 How do you answer "why are you leaving your current role" without sounding negative?

Frame around what you're moving *toward* — broader technical leadership scope, specifically the guild-lead model — rather than what you're moving away from. Even if some frustration is part of the real reason, the interview answer should stay forward-looking; dwelling on what's wrong at your current employer rarely helps and can raise unnecessary flags.

HR-04 Anything worth flagging about location or remote policy for this specific role?

Yes, genuinely worth asking about directly: TCG Digital's major India base is Kolkata (Salt Lake, Sector V), while you're Bangalore-based, and the JD itself doesn't state this role's location or remote/hybrid policy. That's not something to assume either way going into a second round — it materially affects whether an eventual offer is even logistically workable, so it belongs on your questions-to-ask list, not left for after an offer lands.

HR-05 How do you handle a lowball offer without burning the relationship?

Respond with data — market range, current comp — rather than emotion, express genuine continued interest while asking for a revision, and know your own walk-away point in advance so you're not deciding it live under pressure in the moment.

HR-06 What should you have ready for a background verification process?

Accurate, consistent employment dates, titles, and reasons for leaving across your resume, LinkedIn, and whatever you tell HR directly — verification processes commonly cross-check all three, and small inconsistencies, even entirely innocent ones, can slow an offer down or raise unnecessary questions.

HR-07 How do you negotiate title or level if the initial offer undertitles the role relative to your experience?

Point directly to the JD's own stated scope and requirements — 10–15 years, explicit guild leadership responsibility — against the proposed title or band, and ask plainly whether the level reflects the role's actual scope. This is a normal, legitimate negotiation at this seniority, not an awkward overreach.

HR-08 What's a strong answer to "do you have other offers, or where else are you interviewing"?

Honest and brief, without oversharing specifics that could be used to lowball you. Acknowledging you're in active conversations elsewhere, if true, signals market demand without turning the conversation into a negotiation before an offer even exists.

HR-09 How do you handle being asked about a gap or a short stint on your resume, if it applies to you?

Brief, factual, and forward-looking — state plainly what happened without over-explaining or apologizing, then pivot to what you did with the time or learned from the stint. Dwelling on it tends to invite more scrutiny than it resolves.

Round 2 — Closing Strong

A different set of closing questions than round one's — senior and logistics-appropriate, for a conversation that's likely later-stage and higher-altitude.

ASK--01 What should you ask the interviewer in this specific round?

- "How is success in this role actually measured at six months, and at one year?"
- "What does the growth path from Guild Lead look like — is there a defined AVP or Director-equivalent track?"
- "How involved would this role be in client-facing conversations day-to-day, versus purely internal guild work?"
- "Is this role tied to a specific location, or is it remote or hybrid — and if location-tied, which base?"
- "What's leadership's biggest concern about this role six months from now, if it doesn't go the way they hope?"
- "How much budget or investment authority does the guild lead actually have for tooling or headcount asks?"
- "Who would I be working most closely with day-to-day, and how much of my time is expected to go to hands-on work versus coordination?"

ASK--02 With limited time at the end, how do you pick which two or three questions to actually ask?

Prioritize whatever most changes your decision to accept an offer — location and scope questions usually outrank pure curiosity questions at this stage. Read the room too: a tightly scheduled panel earns one or two sharp questions; a more relaxed closing conversation can go longer.

ASK--03 How do you close a senior-round interview strongly, beyond just the questions you ask?

A brief, genuine restatement of why you're specifically excited about this role — not generic enthusiasm — paired with a clear, confident statement that you're interested and ready for next steps. Senior interviews often end anticlimactically simply because the candidate never explicitly closes; don't leave that unsaid.

ASK--04 How do you follow up after this round?

A short thank-you note within 24 hours that references something specific from the actual conversation — not a generic template — reiterating genuine interest. At this seniority it doesn't need to be long; specificity matters more than length.

Quick Reference

L2 Rapid-Fire Answers (ChatGPT-predicted, precise & short) · 11 sections · 111 questions

Guild Leadership & Influence

ChatGPT's predicted L2 list, answered short and precise — sayable in 20-30 seconds, not read from a page. The gold [KEY] badge marks the 10 questions it flagged as highest-probability.

GUIL-01 [KEY] You have 12 delivery pods and none of the engineers report to you. How will you establish SRE standards and get adoption?

Start with one willing pod, prove the standard works there, then expand from evidence, not mandate. Lead with a golden-path template that's easier to adopt than to ignore, publish a short non-negotiables list, and track adoption as a guild-health metric I report upward. Credibility comes from being useful in an incident, not from a policy document.

GUIL-02 [KEY] How would you create the SRE Guild charter for TCG Digital?

One page: purpose, scope (SLOs, incident process, observability, IaC patterns — not day-to-day pod priorities), a small non-negotiables list, the engagement model, and the health metrics I'll report. Drafted after listening to every pod lead first, not before.

GUIL-03 How do you influence a pod lead who doesn't agree with your SRE recommendation?

Ask what their real constraint is before pushing back — usually time or a misread of the risk, not disagreement in principle. Show the specific failure mode the recommendation prevents, offer the lowest-friction path to adopt it, and if they still decline, document the risk and move on rather than escalate immediately.

GUIL-04 [KEY] Tell me about a time you influenced a technical decision without formal authority.

Use a real STAR example from your 14+ years — the interviewer is scoring whether you led with data/credibility, adapted to resistance, and whether the change stuck after you moved on. Pick your story tonight; don't improvise it live.

GUIL-05 [KEY] Two pods want different observability tools — one wants Datadog, one wants Prometheus/Grafana. How do you decide?

I don't force one vendor — I standardize the outcome: every service exposes the four golden signals and pages through the same on-call path, regardless of backend. I'd only push for one tool if there's a real shared cost or compliance driver.

GUIL-06 How do you prevent the SRE Guild from becoming a central bottleneck?

Document everything so I never answer the same question twice, build self-service golden paths instead of gatekeeping, and cultivate an embedded champion per pod so most questions resolve locally without me.

GUIL-07 [KEY] How do you measure whether the SRE Guild is actually successful?

Adoption rate across pods, SLO attainment trend, MTTR trend, and automation/toil-reduction coverage — reported monthly. In a new guild, I'd track adoption and existence metrics first, since there's no baseline yet to show improvement against.

GUIL-08 How would you handle a pod that repeatedly ignores the guild's reliability standards?

Coach first — assume a real constraint, not defiance. If it's a genuine pattern with no constraint behind it, I'd loop in their pod lead, since sustained non-adoption needs the pod lead's buy-in to fix, not authority I don't have.

GUIL-09 How would you contribute to SRE hiring and career development when engineers don't report to you?

A shared SRE competency rubric pod leads use for job descriptions and calibration, a seat on interview panels for SRE-titled roles, and a light mentoring track for pod engineers who want to grow into SRE work.

GUIL-10 How would you mentor engineers across multiple pods without impacting their delivery commitments?

Keep it embedded in real work — pairing during an actual incident or a real PR, not a separate training track competing with their sprint. Rotating guild-sync presentations also builds skills without dedicated time cost.

SLO & Error Budget

SLO-01 How would you define SLOs for a service with no historical reliability data?

Instrument first, observe for a couple of weeks, then set the initial SLO close to what's actually being measured — not an aspirational number. An SLO nobody can hit on day one gets ignored.

SLO-02 How do you choose the right SLIs for a customer-facing application?

Pick what a real user actually feels — availability and latency on the primary journey, like checkout or login — not every metric that's technically measurable. Start narrow.

SLO-03 What happens when a service consumes 80–90% of its error budget?

That's the warning zone, not the trigger — flag it to the pod, tighten release scrutiny, and prioritize reliability work now so it doesn't fully exhaust before the window resets.

SLO-04 Error budget is completely exhausted but Product wants to release an important feature — what do you do?

The pre-agreed policy should trigger: release pauses except reliability work, unless the pod lead and delivery leadership explicitly accept the risk with eyes open. I make the tradeoff visible; I don't unilaterally block a business call that isn't mine alone.

SLO-05 [KEY] How do you use error budgets to make release decisions?

Check burn rate before shipping — healthy budget, ship normally; tight budget, delay, roll out to a percentage, or require explicit sign-off. After deploy, burn rate is often the earliest rollback signal, faster than waiting for a support ticket.

SLO-06 What is burn rate, and how would you implement burn-rate alerts?

The rate you're consuming error budget, not just whether you've breached it. I'd implement multi-window, multi-burn-rate alerts — a fast window catches a sudden outage, a slower window catches a persistent, lower-grade degradation the fast one would miss.

SLO-07 How do you prevent teams from choosing meaningless SLOs?

Anchor every SLO to a real user journey and require it be set from observed data, not a round number. I'd review new SLOs against the guild template mainly checking they'd actually change a release decision if breached.

SLO-08 A shared database causes SLO breaches across five services. Who owns the breach, and what do you do?

The database owner owns the root cause; each affected service owns its own customer-facing SLO and applies local mitigation like retries or caching in parallel. I'd run this as one shared incident, not five separate ones.

SLO-09 How would you roll up SLO health across 12 pods for leadership?

One row per pod: current attainment vs. target, budget remaining, trend arrow, one-line reason if it's red. Leadership needs which engagements are at risk this month, not the PromQL behind it.

SLO-10 How do you know whether an SLO is too strict or too relaxed?

Too strict: budget's constantly exhausted but users aren't actually complaining. Too relaxed: budget's never touched, but support tickets and churn say otherwise. Either signal means revisit the target against real user impact.

Observability

OBS-01 Design an observability standard for all TCG Digital pods.

Standardize the four golden signals as a required contract — same naming, same alert-severity mapping, same on-call routing — regardless of backend. Ship it as a golden-path Helm chart so adopting it is less work than not adopting it.

OBS-02 What are the four golden signals? Give a production example.

Latency, traffic, errors, saturation. Example: checkout p99 under 300ms, error rate under 1% over 5 minutes, request volume baselined so spikes are visible, connection-pool saturation alerting at 80% before it maxes out.

OBS-03 A dashboard is green but customers report checkout failures. How do you investigate?

Check whether the dashboard is measuring the wrong thing — server-side health often looks fine while the failure is client-side: a CDN, a third-party payment API, DNS, or a specific app version. I'd pull real-user monitoring or synthetics from outside the infra.

OBS-04 [KEY] How would you reduce alert fatigue across multiple pods?

Make every alert symptom-based and actionable with a runbook link, split page-worthy from ticket-worthy, and track alert-to-page ratio per pod as a guild health check. Alerts that never lead to action get demoted or deleted, not tolerated.

OBS-05 What makes an alert actionable?

There's something a human can do about it right now, it's tied to real user impact rather than a raw resource number, and it links straight to a runbook — otherwise it's a dashboard metric, not a page.

OBS-06 How do you decide what should page an engineer versus create a ticket?

Page for customer-facing impact happening now that needs action within minutes. Ticket for anything that can wait for business hours or doesn't need a human to intervene immediately.

OBS-07 Metrics are normal, but users report intermittent latency. What do you check?

Look below the aggregate — a p50 hiding a bad p99, a single noisy dependency, client-side or network factors metrics don't capture, or one region or client skewing the average toward normal.

OBS-08 How would you design Prometheus/Grafana dashboards for a critical service?

Golden signals first, drill-down panels for saturation and dependency health second, deploy markers overlaid so changes correlate with symptoms, dashboards defined as code in git so they're reusable as a template.

OBS-09 When would you use metrics vs. logs vs. traces?

Metrics: something's wrong and roughly where — cheap, fast, good for alerting. Logs: why, at the level of one request or error. Traces: where time actually went across services in one request — essential once it's a microservices call chain.

OBS-10 How would you implement distributed tracing across microservices?

OpenTelemetry instrumentation with consistent context propagation across every service boundary, a trace backend like Jaeger or Tempo, and sampling tuned so cost doesn't explode — full sampling on errors, reduced on happy-path traffic.

OBS-11 How do you detect a slow memory leak before it becomes a production incident?

Alert on a sustained upward memory trend over hours or days, not just a saturation threshold — a leak looks fine against a static limit until it isn't. Correlate with deploy markers to catch which release introduced it.

OBS-12 How would you standardise dashboards while allowing pods to use different monitoring tools?

Same principle as the Datadog/Prometheus question — standardize the golden-signal contract and layout convention, not the vendor. Two pods on different backends should both answer "is this healthy" in the same shape.

Incident Management

INC-01 [KEY] Walk me through your first five minutes of a Sev-1 incident.

Confirm severity and declare it formally, get one person driving the fix and one scribing the timeline, check what changed recently — deploys, config, traffic — and open a single incident channel so status lives in one place.

INC-02 A production deployment at 2 AM causes failures across multiple services. What do you do?

Rollback first, root-cause after — at 2am with multiple services down, speed to mitigate beats understanding why. Confirm the rollback actually restored health, then start the postmortem once stable.

INC-03 How do you decide incident severity?

Customer-facing impact and blast radius, not internal inconvenience. Sev1: full outage or critical data issue, page now. Sev2: partial degradation, page with pod lead aware. Sev3/4: workaround exists or no user impact, ticket.

INC-04 What is your role as Incident Commander?

Coordinate, not necessarily fix. Keep one source of truth for status, decide when to escalate, manage stakeholder comms so engineers stay heads-down, make the rollback-vs-continue call, and declare when it's actually resolved.

INC-05 How do you manage stakeholder communication during a major incident?

One person owns comms, separate from the people fixing it. Short updates on a fixed cadence even if the update is "still investigating" — silence is worse than a boring update.

INC-06 How do you decide between rollback and forward fix?

Rollback if the fix is known and fast — usually the default. Forward-fix only if rollback isn't possible or the root cause is unrelated to the last deploy. When in doubt, rollback first and investigate after.

INC-07 The same root cause keeps appearing in incidents. What do you do at Guild level?

A single postmortem action clearly isn't fixing it — pull the pattern across incidents, bring the data to the pod lead, and offer real help: pairing, a focused reliability sprint, or looping in Architecture & Design if it's a design-level gap.

INC-08 How do you ensure postmortem actions are actually completed?

Every action gets an owner and a due date at creation, not "someone should." I'd track open postmortem actions as a guild health metric, and flag a pod carrying too many as a prioritization problem.

INC-09 What makes a good blameless postmortem?

Timeline of events and decisions, not people's names attached to mistakes. Root cause goes past the human action to the systemic gap that allowed it. Action items target process or tooling, not "be more careful."

INC-10 MTTR is increasing even though incident volume is decreasing. What does that tell you?

Probably the remaining incidents are harder — fewer but gnarlier, or knowledge/tooling gaps are slowing diagnosis. I'd break MTTR down by incident type rather than trust the aggregate.

INC-11 How would you redesign on-call for multiple pods?

Each pod owns its own rotation sized to its team, with a standardized handoff and escalation contract guild-wide — including time-zone-aware routing given an onshore/offshore delivery model.

INC-12 How do you prevent engineers from being overloaded by on-call?

Track page volume per person, not just per pod. Enforce a minimum team size before a pod runs its own rotation, and treat a consistently noisy on-call as an alert-quality problem to fix, not a staffing problem.

Kubernetes & Cloud Scenarios

K8S-01 A production pod is in CrashLoopBackOff. Walk me through your investigation.

kubectl describe for events first — image pull failure, OOMKilled, failed probe — then kubectl logs --previous to see the crash before the restart wiped it. Resource limits and probe misconfiguration are the two most common causes.

K8S-02 CrashLoopBackOff started immediately after deployment. What do you check first?

The diff in that deploy — config change, new env var, a dependency the app now expects at startup that isn't there yet. Almost always something in that release, not an unrelated infra issue.

K8S-03 HPA suddenly scales from 5 to 50 replicas. What do you investigate?

Real traffic spike vs. a metrics anomaly vs. a bug causing artificial load — check request volume against the scaling trigger. Also check for a missing maxReplicas ceiling and whether this masks an underlying performance regression.

K8S-04 Pods are healthy but users are getting 502/504 errors. Where do you look?

The layer between the user and the pod — load balancer or ingress config, readiness probe timing (traffic routed before the pod's actually ready), or a timeout mismatch between the proxy and the app.

K8S-05 One Kubernetes node is consuming 100% CPU. How do you troubleshoot?

kubectl top pods on that node to find the offender, check for a missing CPU limit letting one pod starve its neighbors, and check if it's one bad pod vs. genuinely under-provisioned node capacity.

K8S-06 Pods are Pending and cannot be scheduled. What do you check?

kubectl describe pod for scheduling events — usually insufficient resources, a taint/toleration mismatch, or a node selector that can't be satisfied by any current node.

K8S-07 How do you define resource requests and limits?

Requests from real observed usage, not a guess — pull actual data via VPA recommendation mode before setting numbers. Limits high enough to absorb normal spikes but capped to prevent one pod starving its neighbors.

K8S-08 How would you standardise Kubernetes deployments across multiple pods?

A golden-path Helm chart with the guild's non-negotiables baked in by default — resource limits set, probes required, standard labels for cost and observability — so pods extend it instead of reinventing it.

K8S-09 How would you design Kubernetes multi-tenancy for multiple clients?

Namespace-per-pod or per-client baseline with ResourceQuotas and NetworkPolicies enforced. For clients with real compliance needs, dedicated node pools or clusters — a decision framework the guild defines, not a one-size call.

K8S-10 How would you design RBAC for multiple pods sharing a cluster?

Namespace-scoped RoleBindings per pod, guild holds cluster-admin, tied to the org's identity provider rather than Kubernetes-native RBAC as a separate source of truth.

K8S-11 How would you perform a guild-wide Kubernetes upgrade safely?

Staged: non-prod first, one low-risk pod's namespace as canary, then the rest — never everyone at once. Scan for deprecated API usage in advance and communicate the window so no pod is surprised.

K8S-12 How would you provide cost visibility for pods sharing Kubernetes infrastructure?

Enforce cost-attribution labels at the chart level — schema-validated, not just a convention — feeding a chargeback tool like Kubecost, with a per-pod cost dashboard as a standard guild offering.

K8S-13 [KEY] How would you standardise Kubernetes, CI/CD, and IaC together using golden paths?

One paved road, not three separate standards: a Helm chart with guild defaults baked in, wired to a shared CI/CD pipeline template with security gates pre-configured, provisioned by a versioned Terraform module — a new pod gets a compliant, deployable service in one step instead of assembling three standards themselves.

IaC / Terraform / Automation

IAC-01 How would you design reusable Terraform modules for multiple pods?

A small guild-owned set for common patterns — network baseline, node group, database-with-backups — versioned with semantic tags so pods pin a known-good version instead of pulling main.

IAC-02 How do you prevent Terraform from becoming a bottleneck?

Separate state per pod, not one shared file — a monolithic state turns every apply into a blast-radius risk and a lock-contention queue. Guardrails via policy-as-code in CI, not a manual gate I personally clear.

IAC-03 How do you manage Terraform state securely?

Remote backend — S3 plus a lock table, or the cloud-equivalent — encrypted, namespaced per pod and environment, with the backend infrastructure itself guild-owned so pods don't each reinvent it.

IAC-04 How do you handle a dangerous terraform destroy in production?

Prevention first: prevent_destroy lifecycle blocks on critical resources, state locking, destroy requiring a separate deliberate approval step. If it happens anyway, restore from the last known-good state and run it as a real incident.

IAC-05 How would you enforce Terraform standards across pods?

Policy as code — OPA or Sentinel — checked in CI before any apply: no public buckets, mandatory tags, encryption required. Automated guardrails scale; a human review gate on every change doesn't.

IAC-06 What would you automate first when you inherit a high-toil environment?

Whatever the same complaint shows up for across the most teams — that's the highest-leverage target since it justifies guild-level investment. I'd ask engineers directly what they hate doing repeatedly rather than guess.

IAC-07 How do you measure toil?

Roughly, self-reported share of operational time that's manual, repetitive, and automatable — tracked as a trend per pod, not chased for false precision. Google's SRE book definition is the reference: no enduring value once done.

IAC-08 How do you safely implement automated remediation?

Start read-only or dry-run, add a hard blast-radius limit, require it to be reversible, log every action it takes. I wouldn't auto-remediate anything I haven't watched run safely in staging first.

IAC-09 When should you NOT automate something?

When it's rare, when the automation's own failure mode is worse than the manual toil it saves, or when you don't yet understand the problem well enough — automating a bad process just makes it fail faster and less visibly.

IAC-10 How would you build a self-service platform for delivery teams?

Start from the golden-path template pods are already voluntarily pulled toward, then wrap it in a thin self-service layer — a CLI or simple portal — rather than building a platform speculatively before there's proven demand.

CI/CD & Release Engineering

CICD-01 How would you standardise CI/CD across multiple pods using different tools?

Don't force a tool migration without a real driver — standardize the CD/GitOps model (ArgoCD) since consistent deployment state pays off in incident response, but let Jenkins vs. GitLab CI vary by pod if that's just existing toolchain.

CICD-02 How do you implement quality gates in Jenkins?

Pipeline stages that fail the build on threshold breach — test coverage, static analysis severity, vulnerability severity — as a shared Jenkinsfile template pods extend, not something each pod configures inconsistently.

CICD-03 How do SonarQube and Trivy fit into a production pipeline?

SonarQube for code quality and static analysis gates pre-merge; Trivy for container and dependency vulnerability scanning pre-deploy. Both as blocking CI stages with severity thresholds, not optional reports nobody reads.

CICD-04 When should a pipeline fail because of a vulnerability?

Critical or high severity with a known exploit path blocks the build, always. Medium/low can warn and ticket rather than block, or minor CVEs turn every release into a fire drill.

CICD-05 How would you implement progressive delivery?

Canary via Argo Rollouts with automated analysis gates tied to golden-signal metrics — promote automatically if healthy, roll back automatically if error rate or latency breaches threshold.

CICD-06 Canary vs. blue-green deployment — when would you use each?

Canary for gradual, metric-gated exposure when you can tolerate some users on the old version briefly — lower infra cost, slower full rollout. Blue-green for an instant, clean cutover when you can afford two full environments — faster switch, higher cost.

CICD-07 How would you automatically roll back a failed deployment?

Health checks and golden-signal thresholds wired into the deployment controller — Argo Rollouts' analysis step — so a breach triggers automatic rollback without a human needing to notice first.

CICD-08 How do you prevent a bad deployment from exhausting the error budget?

Canary plus automated rollback tied to burn-rate, not just static error-rate thresholds — catch the bad deploy in the canary slice before it's serving full traffic, so the budget hit is contained and brief.

CICD-09 How would you migrate a legacy application from manual deployment to CI/CD?

Incrementally — automate build and test first, keep manual deploy; then automate deploy to non-prod; then prod behind a manual approval gate initially; remove the gate once trust is earned. Never a big-bang cutover.

DR / Chaos / Resilience

DR-01 [KEY] Design DR for a Tier-1 multi-region application.

Define RTO/RPO first from actual business impact, not a guess. Active-passive by default unless the RTO genuinely can't tolerate failover time; data replication matched to the RPO; a tested, not just documented, failover runbook.

DR-02 Active-active vs. active-passive — how do you choose?

Active-passive is simpler and cheaper but costs failover time and leaves the standby's readiness untested until needed. Active-active gives faster failover and continuously validates both regions, at real data-consistency complexity. Default to passive unless the SLO can't tolerate the failover time.

DR-03 How do you determine RTO and RPO?

From actual business impact of downtime and data loss, in conversation with the business or client side — not an engineering guess. Then reality-check whether the current architecture can actually hit those numbers.

DR-04 How do you prove that DR actually works?

Run the failover for real, on a cadence — at least annually, more for client-critical services — and measure actual RTO against the documented target. An untested DR plan is just an assumption.

DR-05 What chaos experiments would you run for a Kubernetes application?

Pod-delete first — lowest risk, tests basic resilience. Then network latency or partition injection, then a node drain, then a dependency failure simulation — escalating blast radius only after each prior experiment passes cleanly.

DR-06 How would you test a regional failure?

A controlled failover game day — redirect traffic away from a region deliberately, in a maintenance window initially, with an abort trigger — and measure whether golden signals hold within SLO in the surviving region.

DR-07 How do you safely introduce chaos engineering into production?

Staging first, on a willing pilot pod, with a hard blast-radius limit and automatic abort condition — never a skeptical pod's first chaos experiment live in production with no track record yet.

DR-08 What happens if your database cannot fail over?

That's your real RTO ceiling regardless of what the app tier can do — I'd surface that gap explicitly rather than let a DR plan claim an RTO the database can't support, and prioritize fixing that dependency first.

DR-09 How do you validate that application dependencies also meet the required RTO?

Map every hard dependency's own RTO/RPO, not just the primary service's — a Tier-1 app with a Tier-3 dependency has a Tier-3 real RTO. Chase the weakest link in the chain.

Capacity / Cost / Performance

CAP-01 How do you perform capacity planning for a production service?

Trend current utilization against historical growth, add a safety margin sized to how volatile that specific service's traffic is, reviewed quarterly rather than annually given how fast engagements shift.

CAP-02 Traffic is expected to increase 3x in six months. How do you prepare?

Load test at 3x today to find the actual breaking point, not assume linear scaling — databases and stateful components often don't scale linearly. Identify the first bottleneck, fix it, then retest, iterating rather than guessing.

CAP-03 CPU is only 30%, but latency is increasing. What could be happening?

CPU isn't the bottleneck — check saturation elsewhere: connection pool exhaustion, lock contention, a slow downstream dependency, disk I/O, or GC pauses.

CAP-04 How do you identify the actual bottleneck in a distributed system?

Trace a slow request end-to-end rather than guessing from aggregate metrics — distributed tracing shows exactly where time is spent across the call chain, usually not where intuition points first.

CAP-05 How do you optimise Kubernetes costs?

Right-size from real VPA/usage data, not a guess. Clean up orphaned resources and idle load balancers. Use spot or reserved capacity for steady-state baseline load, kept separate from bursty capacity.

CAP-06 How would you identify over-provisioned infrastructure?

Compare requested/allocated resources against actual usage over a real window, per service — anything consistently well under its request is a right-sizing candidate, surfaced through a per-pod cost dashboard.

CAP-07 How do you balance reliability and cloud cost?

Never cut the margin that's actually protecting the SLO just to trim spend — frame any cost change against its reliability impact explicitly, as one joint number, not two separate reviews that never talk.

CAP-08 How would you present a capacity gap to Delivery leadership?

Business terms: what breaks and when if we don't invest, translated to client or revenue risk, backed by the trend data — not just "we need more infrastructure."

Architecture & NFR

ARCH-01 How do you bring SRE requirements into architecture/design before production?

A seat at the design review, not a post-launch audit — non-functional requirements reviewed alongside functional ones before anything's built, since retrofitting reliability afterward is far more expensive.

ARCH-02 What NFRs would you define for a Tier-1 service?

Availability/SLO target, latency budget, RTO/RPO, scalability ceiling, and security/compliance baseline — each with a number, not a vague "highly available" nobody can test against.

ARCH-03 How do you design for graceful degradation?

Identify the core journey that must survive versus nice-to-have features that can shed under load — cache stale-but-served over a hard failure, feature-flag non-critical calls off automatically under pressure.

ARCH-04 How do you design a service that can tolerate dependency failure?

Timeouts and circuit breakers on every external call, a fallback or cached response where sensible, and bulkheading so one dependency's failure can't exhaust the pool shared with everything else.

ARCH-05 How would you design retry, timeout, and circuit-breaker policies?

Timeouts tighter than the caller's own SLO budget. Retries with exponential backoff and jitter, capped low. Circuit breaker trips after a threshold of failures and fails fast instead of piling up retries against a downed service.

ARCH-06 **How do you prevent retry storms?**

Jittered exponential backoff so retries don't synchronize, a retry budget or cap per request, and circuit breakers that fail fast instead of every caller hammering a struggling downstream service at once.

ARCH-07 **How do you design for regional failure?**

Stateless services that can run in any region, data replication matched to your RPO, and DNS/traffic-manager failover tested regularly — the same active-active/passive tradeoff as DR, at the architecture layer.

ARCH-08 **How do you review an architecture from an SRE perspective?**

Where's the single point of failure, what's the blast radius if this component fails, can it be observed and paged on, and what's the actual RTO if it goes down — asked explicitly in the design review, not assumed to be someone else's job later.

Leadership & Behavioural

BEHA-01 **Tell me about your most difficult production incident.**

Pull a real one from your 14+ years — lead with what changed systemically afterward, not just the incident drama. That's the part interviewers probe on follow-up.

BEHA-02 **Tell me about a time you disagreed with an architect or senior engineer.**

Real example, STAR — the interesting part is how you raised it (data, not just opinion) and what happened, including if you were wrong. A mixed track record told honestly lands better than claiming you were always right.

BEHA-03 **Tell me about a time you influenced someone without authority.**

Same story bank as the Guild Leadership version of this question — use one real example consistently rather than improvising a different one each time it's asked.

BEHA-04 **Tell me about a failed automation or reliability initiative.**

Real example — the honest answer includes what you'd do differently, not just what went wrong. Zero failures either means you haven't tried enough, or isn't fully honest.

BEHA-05 **Tell me about a time you reduced operational toil.**

Real example with a concrete before/after — what was manual, what you automated or self-serviced, and the time actually reclaimed.

BEHA-06 **Tell me about a time you improved alert quality.**

Real example — ideally a before/after on alert-to-page ratio or false-positive rate, not just "we added more alerts."

BEHA-07 **How do you handle resistance from developers toward SRE practices?**

Ask what their actual objection is before defending the practice — often it's proportionate friction on a low-risk change, and they're right. If the practice is genuinely justified, show the specific failure mode it prevents rather than asserting the standard's importance.

BEHA-08 How do you mentor a strong engineer who disagrees with you?

Engage with their actual argument on its merits, not on seniority — if they're right, say so and adjust. If I still disagree after hearing them out, I'll say why clearly rather than pulling authority I don't have as guild lead anyway.

BEHA-09 How do you prioritise when five pods need your help simultaneously?

Objective criteria, not whoever asked loudest: client-facing risk severity, blast radius, and leverage — a small nudge that fully unblocks a nearly-there pod often beats a big lift on a far-behind one. I'd make the tradeoff visible to delivery leadership.

BEHA-10 How do you balance delivery pressure with reliability?

Make the trade-off visible instead of absorbing it silently — show the actual risk being taken on, let delivery leadership decide with the number in front of them. The guild's job is a fast path that's also safe, not a roadblock or a rubber stamp.